



Article

Compensating PI Controller's Transients with Tiny Neural Network for Vector Control of Permanent Magnet Synchronous Motors

Martin Joel Mouk Elele ^{1,2}, Danilo Pau ^{2,*} , Shixin Zhuang ³ and Tullio Facchinetti ⁴

¹ Department of Physics, Università di Milano Bicocca, 20126 Milan, Italy; m.martinjoel@campus.unimib.it

² STMicroelectronics, System Research and Applications, 20864 Agrate Brianza, Italy

³ MathWorks, Natick, MA 01760, USA

⁴ Department of Industrial and Information Engineering, Università di Pavia, 27100 Pavia, Italy; tullio.facchinetti@unipv.it

* Correspondence: danilo.pau@st.com

Abstract: Recent advancements in neural networks (NNs) have underscored their potential for deployment in domains that demand computationally intensive operations, including applications on resource-constrained edge devices. This study investigates the integration of a compact neural network, TinyFC, within the Field-Oriented Control (FOC) framework of a Permanent Magnet Synchronous Motor (PMSM). While proportional–integral (PI) controllers remain a widely adopted choice for FOC due to their simplicity, their performance can degrade significantly under high-frequency speed transitions, where nonlinear dynamics introduce notable inaccuracies. The TinyFC model complements the PI controller by learning the intrinsic dependencies within the control loops and generating corrective signals to alleviate these inaccuracies. To ensure practical implementation, TinyFC underwent extensive optimization procedures, incorporating advanced techniques such as hyperparameter tuning, pruning, and 8-bit quantization. These measures successfully reduced the model's computational overhead while preserving predictive accuracy. Simulation results demonstrated that embedding TinyFC within the FOC framework substantially reduced overshoot, with the pruned TinyFC entirely eliminating overshoot when integrated into the speed control unit. These findings highlight the feasibility of employing lightweight neural networks for real-time motor control applications, establishing a foundation for more efficient and precise control strategies in edge automotive and industrial systems.



Academic Editor: Ming Yao

Received: 15 March 2025

Revised: 11 April 2025

Accepted: 16 April 2025

Published: 18 April 2025

Citation: Elele, M.J.M.; Pau, D.; Zhuang, S.; Facchinetti, T. Compensating PI Controller's Transients with Tiny Neural Network for Vector Control of Permanent Magnet Synchronous Motors. *World Electr. Veh. J.* **2025**, *16*, 236. <https://doi.org/10.3390/wevj16040236>

Copyright: © 2025 by the authors. Published by MDPI on behalf of the World Electric Vehicle Association. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: tiny neural networks; field-oriented control; permanent magnet synchronous motor; proportional–integral controller; hyperparameter optimization; model pruning; quantization; real-time control systems

1. Introduction

The rapid evolution of computational technologies has ushered in a transformative era for Artificial Intelligence (AI), enabling its application across an expansive array of domains. Today, AI drives innovations ranging from healthcare monitoring [1,2] and diagnostics to natural language processing (NLP), computer vision (CV), autonomous driving, and intelligent environments [3]. Despite the promise of these advancements, they come with substantial trade-offs. Training and deploying AI models typically demand vast computational resources, elevating both hardware and cloud computing costs that continue to grow as cloud usage scales with the number and complexity of executed processes [4]. Furthermore, the reliance on high-performance tensor processing units (TPUs) and other

specialized hardware escalates energy consumption, thereby exacerbating environmental sustainability concerns tied to the ongoing dependency on non-renewable energy sources.

To address these challenges, Edge AI—previously referred to as Tiny Machine Learning (TinyML)—has emerged as a groundbreaking paradigm. Edge AI enables the deployment of advanced AI algorithms on ultra-low-power devices, such as microcontrollers (MCUs), with energy consumption measured in milliwatts (mW) or microwatts (μ W) [5]. Decades of innovation in embedded systems have facilitated the creation of highly energy-efficient, adaptive devices, which are now integral to applications spanning smart cities, agriculture, healthcare, and logistics [6]. Edge AI builds on this foundation, offering scalable solutions that circumvent the architectural and energy constraints inherent to traditional machine learning (ML) systems. By leveraging minimal memory—often less than 256 KiB—and relatively low computing frequencies (40–400 MHz), Edge AI facilitates the execution of sophisticated algorithms on resource-constrained devices [7–9].

The versatility of Edge AI holds transformative implications for various industries. Within manufacturing, it enables real-time anomaly detection by continuously monitoring parameters such as sound and vibration, thereby minimizing downtime and enhancing operational efficiency. Edge AI also addresses the latency and connectivity challenges inherent in cloud-based AI by enabling real-time on-device image and speech recognition—even in offline scenarios. This capability is particularly beneficial in areas with limited internet access or applications requiring low-latency decision-making, such as autonomous systems and healthcare monitoring. Moreover, by processing data locally, Edge AI inherently complies with data protection regulations, offering robust solutions to privacy and security concerns [6].

In parallel, the global shift toward sustainable mobility has gained urgency in response to rising pollution levels and the depletion of fossil fuel reserves. Electric vehicles (EVs) have emerged as a promising alternative, with Permanent Magnet Synchronous Motors (PMSMs) playing a pivotal role in their drive systems. Renowned for their high efficiency, low inertia, compact design, and superior dynamic response, PMSMs align with the demanding performance criteria of automotive, industrial, and aerospace sectors.

PMSMs are widely recognized for their high torque and power density, making them indispensable across automotive, industrial, naval, and aeronautic domains, where precision control, compactness, and reliability are paramount [10]. These motors consist of a stator with windings and a rotor embedded with permanent magnets. Synchronization at synchronous speed is achieved through the interaction between the stator's rotating magnetic field and the rotor's fixed magnetic field [11]. This design merges the strengths of induction and brushless direct current (DC) motors, delivering superior power density, efficiency, and torque characteristics, which establish PMSMs as the optimal choice for advanced industrial and automotive systems.

Field-Oriented Control (FOC) is a widely employed strategy to optimize PMSM performance [12]. FOC facilitates precise torque and speed regulation across the motor's operational range, enhancing efficiency and responsiveness. While proportional–integral (PI) and proportional–integral–derivative (PID) controllers are commonly utilized within FOC frameworks due to their simplicity and robustness, PI controllers are often hindered by their susceptibility to parameter variations and external disturbances [13]. Variations in electrical parameters—such as resistance and inductance—caused by temperature fluctuations and wear introduce uncertainties that compromise the controller's performance.

Advanced control algorithms, including H_∞ -synthesis [12] and Model Predictive Control (MPC), have been developed to enhance robustness. Although effective, these methods significantly increase computational complexity, resulting in unacceptable latencies on resource-constrained MCUs [14]. Despite their limitations, PI controllers remain

widely adopted in industrial motor control due to their ease of implementation and their avoidance of the noise amplification issues associated with the derivative components of PID controllers.

This paper proposes a low-latency control strategy for PMSMs that incorporates a compact neural network (NN) within the FOC framework. The NN, implemented as a lightweight feed-forward network (TinyFC), enhances the responsiveness and precision of motor control systems. Designed for resource-constrained MCUs, TinyFC delivers improved performance while adhering to stringent power and memory constraints set by the FOC framework. By integrating the NN corrective actions into the FOC loop, the system achieves superior responsiveness and accuracy compared to conventional PI-based methods. This hybrid FOC-NN architecture is anticipated to markedly enhance the dynamic performance of PMSM control, addressing the rigorous demands of modern electric drive systems [15]. Figure 1 illustrates the workflow for designing and deploying TinyFC within the FOC framework. Simulation evaluations across two challenging scenarios and MCU implementations underscore the efficacy of the proposed approach, highlighting its potential to address real-world challenges in EV control applications.

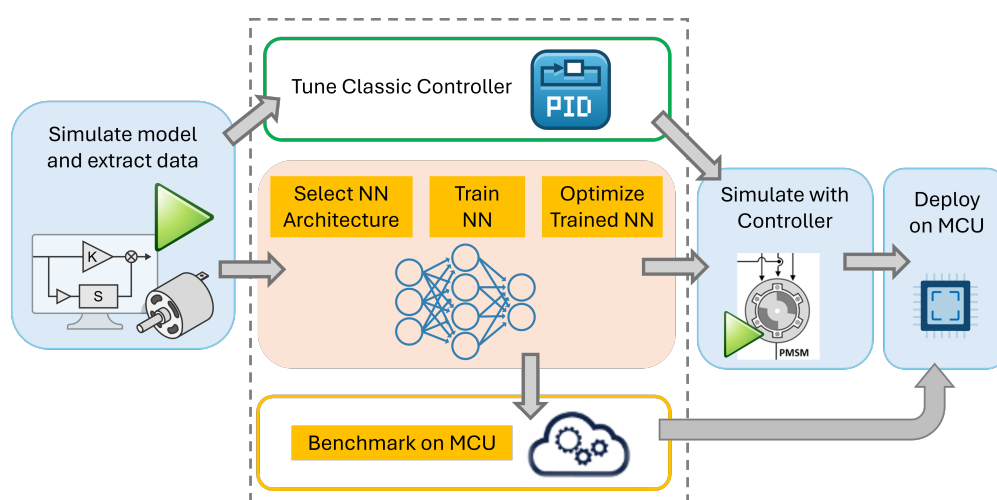


Figure 1. Workflow adopted for deploying NN-augmented FOC.

The rest of this paper is structured as follows: Section 2 reviews the related work in the domain of frame transformations and vector control strategies, emphasizing the limitations of conventional control. Section 3 presents the motor control setup in the simulation environment, with Section 4 focusing on the challenging scenarios (use-cases) presented to the control structure. Sections 5 and 6 cover the devised approaches to enhance PI-based control in both the speed and current control units, dataset creation, NN architecture, training, and optimization techniques for deployment on MCUs. Section 7 shows and discusses the performance improvements achieved by the proposed neural network, alongside its contribution to the existing control strategies.

2. Related Knowledge

The operation of a PMSM is governed by the interaction between the stator's rotating magnetic field and the rotor's constant magnetic field. When a three-phase alternating current (AC) is applied to the stator windings, a rotating magnetic field is generated. This field induces a force on the rotor magnets, prompting the rotor to rotate synchronously with the stator's magnetic field. Two principal categories of control strategies are employed for PMSMs: scalar control and vector control.

Scalar control represents the most fundamental approach, focusing exclusively on the magnitude of control variables such as voltage, current, or frequency. Typically imple-

mented in open-loop configurations, scalar control is suited for applications with relatively low dynamic requirements. Conversely, vector control is a more advanced methodology that offers the precise regulation of both the amplitude and phase of stator currents. By leveraging the real-time feedback of rotor position and speed, vector control enables highly dynamic performance and efficient torque regulation through the application of space vector principles. As highlighted in Section 1, this study centers on the enhancement of FOC for PMSMs, a widely adopted vector control strategy. The remainder of this section elaborates on the necessity of frame transformations, the methodologies employed, and related techniques applicable to PMSM control systems.

The effective analysis and control of PMSMs necessitate the transformation of three-phase signals from the stationary abc frame (stator reference frame) into a more manageable, rotating dq frame (rotor reference frame). This transformation is indispensable due to the inherent complexities associated with controlling PMSMs directly in the abc frame. The sinusoidal nature and continuous variation of the three-phase currents and voltages within the abc frame introduce substantial challenges for real-time motor control. The control system must simultaneously manage these dynamic signals, increasing computational and operational complexity. By transitioning to the dq frame, which rotates synchronously with the rotor's magnetic field, the sinusoidal motor currents are effectively converted into constant values under steady-state operation. This simplification facilitates the implementation of advanced control strategies, streamlining the regulation of critical parameters such as torque and flux. These parameters are instrumental in ensuring efficient motor operation [16].

2.1. Frame Transformation

In the abc reference frame, the PMSM operates as a three-phase system with currents as listed in Equation (1):

$$\begin{aligned} i_a &= |\bar{I}| \cos(\omega_e t) \\ i_b &= |\bar{I}| \cos(\omega_e t - \frac{2\pi}{3}) \\ i_c &= |\bar{I}| \cos(\omega_e t + \frac{2\pi}{3}) \end{aligned} \quad (1)$$

where i_a , i_b , and i_c are stator currents, ω_e is the electrical angular velocity, and $\bar{I}$ is the phasor representation of the current in the stator frame. Due to the time dependence in the abc reference frame, the Clarke Transform is used to convert the three-phase currents to a two-dimensional stationary reference frame (α, β) [17], where i_0 is the zero sequence current of the neutral conductor, and K_{Clarke} is the Clarke transformation matrix as formulated in Equation (2):

$$\begin{bmatrix} i_\alpha \\ i_\beta \\ i_0 \end{bmatrix} = K_{Clarke} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \quad (2)$$

The Park Transform further converts the stationary $\alpha\beta$ frame to a rotating dq frame as shown in Equation (3):

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = K_{Park} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} \quad (3)$$

where K_{Park} is the transformation matrix with respect to the electrical angle. The dq frame rotates with the rotor, simplifying control as i_d and i_q become DC quantities under steady-state conditions, with electromagnetic torque proportional to i_q .

PMSMs are powered by inverters, which convert DC to three-phase AC. The inverter uses Pulse Width Modulation (PWM) to generate the required voltage and current for the motor. As shown in Figure 2, the inverter consists of three half-bridge units where the

upper and lower switches are controlled complementarily, meaning when the upper switch is turned on, the lower one must be turned off, and vice versa [10,18].

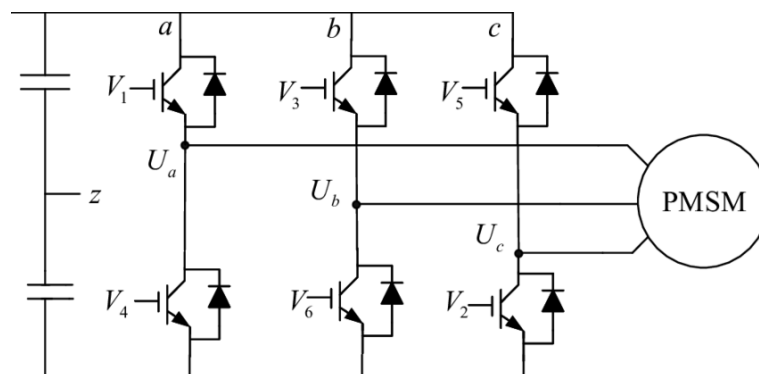


Figure 2. Three-phase inverter circuit connected to PMSM.

Generally, for a surface PMSM (SPMSM), the stator current equations in the dq reference frame can be simplified as shown in Equation (4) [19]:

$$\begin{aligned}\omega_e &= \frac{P}{2} \omega_m \\ \frac{di_d}{dt} &= \frac{1}{L_d} v_d - \frac{R}{L_d} i_d + \frac{L_q}{L_d} \frac{P}{2} \omega_m i_q \\ \frac{di_q}{dt} &= \frac{1}{L_q} v_q - \frac{R}{L_q} i_q - \frac{L_d}{L_q} \frac{P}{2} \omega_m i_d - \frac{\lambda_{pm} \frac{P}{2} \omega_m}{L_q} \\ T_e &= \frac{3}{2} \frac{P}{2} [\lambda_{pm} i_q + (L_d - L_q) i_d i_q]\end{aligned}\quad (4)$$

where we have the following:

- ω_m : mechanical angular velocity;
- P : number of poles;
- L_d, L_q : d- and q-axis inductances;
- R : stator resistance;
- v_d, v_q : d- and q-axis voltages;
- i_d, i_q : d- and q-axis currents;
- λ_{pm} : permanent magnet flux linkage;
- T_e : electromagnetic torque.

2.2. Vector Control

PI-based FOC is extensively utilized for PMSM control, owing to its simplicity and effectiveness. This approach employs PI controllers to dynamically adjust input voltages using real-time feedback from current and speed sensors. Despite its prevalence, achieving the optimal tuning of PI gains (K_p, K_i) under diverse operating conditions remains a significant challenge. This difficulty arises from the inherent complexity of the Multiple-Input Multiple-Output (MIMO) nature of PMSMs, which contrasts with the simpler Single-Input Single-Output (SISO) configuration of PI controllers.

To overcome these limitations, the control problem can be reformulated using robust methodologies such as μ -synthesis [12]. μ -synthesis enhances system performance by minimizing sensitivity to uncertainties. For practical implementation, the complexity of μ -synthesis is often reduced to H_∞ -synthesis, which optimizes the system's performance by minimizing the H_∞ -norm of the closed-loop transfer function. This simplification retains robustness while achieving computational efficiency.

MPC provides a promising alternative to PI-based FOC [18], leveraging predictive algorithms to optimize control actions. MPC formulates a control strategy based on a discrete-time state-space model, enabling the anticipation of system behavior and the proactive adjustment of control inputs. The mathematical foundation of MPC, presented in Equation (5), underscores its capability to handle multivariable and constrained optimization scenarios, making it well suited for the dynamic requirements of MIMO systems:

$$\begin{aligned}x(t+1) &= A \cdot x(t) + B \cdot u(t) \\y(t) &= C \cdot x(t) + D \cdot u(t)\end{aligned}\quad (5)$$

where $x(t)$ is the state vector, $u(t)$ is the control input, and $y(t)$ is the output. The cost function minimizes tracking errors and control effort, which in the case of PMSMs is formulated as shown in Equation (6):

$$g(t) = \left(i_d^* - i_d^p(t+1)\right)^2 + \left(i_q^* - i_q^p(t+1)\right)^2 \quad (6)$$

where i_d^* , i_q^* are reference currents, and $i_d^p(t+1)$, $i_q^p(t+1)$ are predicted currents. The predictive model thus takes the form described in Equation (7):

$$\begin{aligned}i_d^p(t+1) &= \left(1 - \frac{T_s R}{L_s}\right) i_d(t) + T_s \omega_e i_q(t) + \frac{T_s}{L_s} v_d \\i_q^p(t+1) &= \left(1 - \frac{T_s R}{L_s}\right) i_q(t) - T_s \omega_e i_d(t) + \frac{T_s}{L_s} v_q - \lambda_{pm} \omega_e T_s\end{aligned}\quad (7)$$

with T_s as the sampling period, and R and L_s as the stator resistance and inductance, respectively. MPC integrates system constraints directly into the optimization, ensuring robust and efficient control for PMSMs [10].

2.3. Limitations in Known Control

While PI controllers offer a straightforward structure for motor control, their application within FOC for PMSMs is accompanied by notable limitations and challenges. Designed primarily for linear systems, PI controllers are ill equipped to handle the nonlinearities and dynamic complexities inherent to PMSMs, particularly during high-frequency speed transitions or disturbances, such as rapid variations in load torque. Within an FOC scheme, the simplicity of PI controllers becomes a constraint, as they cannot effectively manage cross-interactions between the d -axis and q -axis currents—a fundamental aspect of PMSM operation. These nonlinear dynamics are exacerbated during high-speed transitions by phenomena such as the following:

- Magnetic saturation;
- Cross-coupling between the d -axis and q -axis;
- Variations in back-EMF.

As a result, the PI controller's inability to dynamically adapt to such conditions leads to performance degradation. Additionally, conventional anti-windup mechanisms, implemented to prevent integrator saturation, often introduce further nonlinear behaviors, complicating the control process. This becomes particularly critical when operating under conditions far removed from the nominal parameters, where the system's response may become unstable. Compounding these challenges is the presence of model uncertainties during the design and deployment of PMSMs, which can significantly degrade PI controller performance. For instance, discrepancies between the actual and modeled plant dynamics—illustrated in Equation (8)—pose substantial difficulties:

$$G_{\text{actual}}(s) = G(s) + \Delta G(s) \quad (8)$$

where $\Delta G(s)$ represents deviations from the modeled plant transfer function $G(s)$. These inaccuracies are further intensified by the fixed-parameter nature of PI controllers, which restrict their ability to adapt to dynamic system variations, thereby compromising stability and overall performance.

In terms of accuracy within the control loop, disturbances or noise significantly affect the output of the PI controller. This impact is captured by the sensitivity function $S(s)$, which characterizes the system's responsiveness to external disturbances as shown in Equation (9):

$$S(s) = \frac{1}{1 + C(s)G(s)} \quad (9)$$

where $C(s)$ denotes the Laplace-domain transfer function of the controller. For the system to maintain accuracy and stability, $S(s)$ must remain well tuned across all frequencies [20]. However, abrupt speed changes frequently induce large sensitivity gains at both low frequencies (slow response to load torque) and high frequencies (noise amplification), reflecting poor disturbance rejection capabilities. Consequently, the PI controller is unable to effectively adapt to these dynamics, further impairing control precision.

Though the enhancement of PI-based control using H_∞ and μ -synthesis does offer enormous benefits in the closed loop, the practicality of this approach remains quite challenging with regards to memory constrained devices. This is because solving Riccati equations or Linear Matrix Inequalities [21] in real-time poses a significant challenge for hardware deployment, making it impractical for most hardware setups due to the computational delays introduced when solving the optimization problem. The predictive nature of the MPC controllers gives them an advantage over the PI controllers. This is because MPC predicts the system's future behavior and adjusts control inputs accordingly, while considering complex control goals and constraints [17]. As shown by the computational complexity of the optimization problem in Equation (10),

$$\min_u \sum_{k=0}^N \left(x_k^T Q x_k + u_k^T R u_k \right) \quad (10)$$

where we have the following:

- x_k is the system state at time k ;
- u_k is the control input at time k ;
- Q and R are weighting matrices that determine the trade-off between state tracking and control effort.

Solving this optimization problem becomes a significant drawback for real-time processes [14]. This is especially true in systems with low time constants such as electric drives, where rapid decisions are required. In such systems, small controller cycle times are needed, a requirement which cannot be met by the optimization base of the MPC, resulting in a high computational burden. Suboptimal approaches such as Finite Control Set MPC or explicit MPC are often used to reduce this complexity; however, these approaches come with a trade-off in accuracy.

Traditional forecasting architectures, including feed-forward neural networks (NNs) and recurrent models like Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), also face significant limitations. These models typically involve numerous hidden layers and neurons (with recurrent models effectively expanding hidden layers over time), substantially increasing computational complexity [22]. For resource-constrained devices, this complexity becomes a critical drawback. Furthermore, the performance of NNs heavily depends on factors such as architecture, parameter count, and activation functions, none of which guarantees effectiveness or robust overfitting avoidance.

3. System Design

This paper brings forth a comparative study of a PI-based FOC and an NN-enhanced PI control, taking into account accuracy, computational complexity, memory footprint and deployability on MCUs. The FOC for PMSM simulation environment is modeled in Simulink with a dataset collected from the simulation environment to train the devised NN. The FOC block used in this work is based on the Simulink model proposed by Juan Camilo Nustes et al. [23] which relies on three PI controllers and used the FOC algorithm provided by STMicroelectronics for its P-Nucleo Series boards, for deployability testing.

Setup and Hardware

The control system, illustrated in Figure 3, comprises three primary subsystems: input (green), FOC (blue), and motor/inverter (gray). The Input subsystem generates reference speed commands, which serve as target signals for the PMSM during the control sequence. These reference signals are forwarded to the FOC subsystem, which is divided into two hierarchical units:

- **Speed control unit.** The speed control unit employs a PI controller to compute the reference quadrature current i_q^{ref} based on the speed error, defined as the deviation between the measured speed obtained from the flux observer and the reference speed. To ensure maximum torque per ampere (MTPA), the reference direct current i_d^{ref} is consistently set to zero. The flux observer integral to the FOC subsystem estimates rotor position θ_m and mechanical angular velocity ω_m using stator currents i_α , i_β and voltages v_α , v_β . The rotor flux linkage is derived from these parameters, enabling the accurate computation of rotor position and speed. The rotor's electrical angle θ_e is converted into mechanical angle θ_m via the motor's pole pairs. A low-pass filter smooths noise in the computed angular speed, ensuring a reliable estimate.
- **Current control unit.** The current control unit comprises two PI controllers that regulate the i_d and i_q currents by minimizing errors between the measured and reference currents. The PI controllers output voltages in the d -axis v_d and q -axis v_q , which are constrained within predefined operational limits using a direct-quadrature (DQ) limiter to prevent over-voltage or over-current scenarios [24]. These limited voltages are transformed into α - β voltages via the inverse Park transform and serve as inputs for the Space Vector Pulse Width Modulation (SVPWM) block. The SVPWM generates the three-phase inverter switching signals, producing the v_a , v_b , and v_c voltages required to drive the PMSM.

The estimator calculates rotor flux linkage and position using α - β frame measurements, while the look-up table (LUT) precomputes trigonometric values, reducing computational complexity and enhancing real-time control performance. Finally, in the motor/inverter, the inverter drives the motor with precise electromagnetic control, with voltages obtained via Clarke and Park transformations. The feedback currents are then compared with reference currents to ensure closed-loop precision in motor operation.

In this study, the FOC setup is modeled in Simulink 2024b based on the BR2804-1700KV-1 motor and the X-NUCLEO-IHM07M1 inverter board from the STM32 Nucleo P-NUCLEO-IHM001 kit [25] provided by STMicroelectronics, with the characteristics listed in Table 1. The BR2804-1700KV-1 motor, with a nominal voltage of 11.1 volts (V), a maximum current of 5 amperes (A), seven pole pairs, and a peak speed of 19,000 revolutions per minute (RPM), is chosen for its precision, high torque, and lightweight properties. The motor's nominal voltage aligns with the inverter's 8–48 V range, and its current specifications are compatible with the inverter's peak capacity of 2.8 A per phase. This setup facilitates the rigorous analysis of the motor's dynamic response to control inputs

under varied scenarios, leveraging the simulation environment as a virtual testbed for the iterative development and refinement of control algorithms.

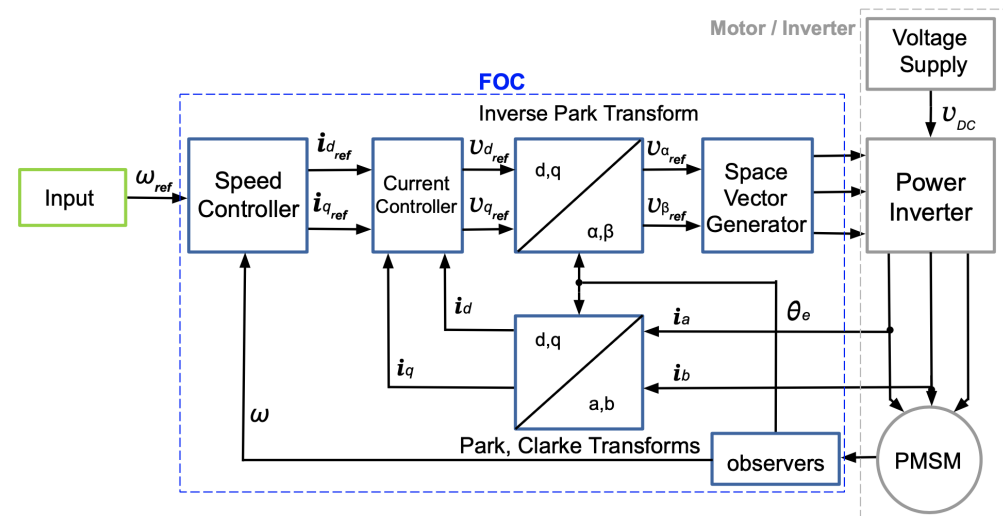


Figure 3. Block diagram of the FOC and motor systems.

Table 1. Main characteristics of the motor and inverter model used as reference in the simulation.

	Parameter	Attribute
Motor	Nominal Voltage	11.1 V DC
	Maximum Current	5 A
	Pole Pairs	7 pole Pairs
	Maximum Speed	19,000 RPM
Inverter	Driver	3-Phase driver
	Nominal Voltage	8 V to 48 VDC
	Output peak current	2.8 A
	Compatibility	STM32 Nucleo Boards

The dataset collected through this setup mimics the motor's response to diverse input speed profiles, particularly nonlinear ones, providing a robust platform for evaluating alternative control strategies. A lightweight neural network (TinyFC) is developed as a nonlinear control mechanism to surpass the performance of traditional PI controllers under these specific conditions. This integrated simulation environment underscores the potential of advanced control algorithms in achieving precise, real-time motor operation.

4. Test Cases

This study evaluates the limitations of PI-based control by introducing two challenging use cases, each paired with a test case designed to stress the controller under demanding conditions. The test cases are intentionally developed to evaluate the TinyFC model's ability to generalize and capture the latent dependencies critical for minimizing overshoot in PMSM control. The model not trained with data from the extra test cases will clearly showcase its ability to understand the value by which the PI prediction is deviating, and correct it.

- The first use case involves two speed variations per second (see Figure 4).
- The second one pushes the system with ten variations per second (see Figure 5).

Custom input signals, generated using Simulink's "Signal Builder" block, simulate high-frequency transitions with varying magnitudes and durations, mimicking real-world scenarios to test the controller's responsiveness.

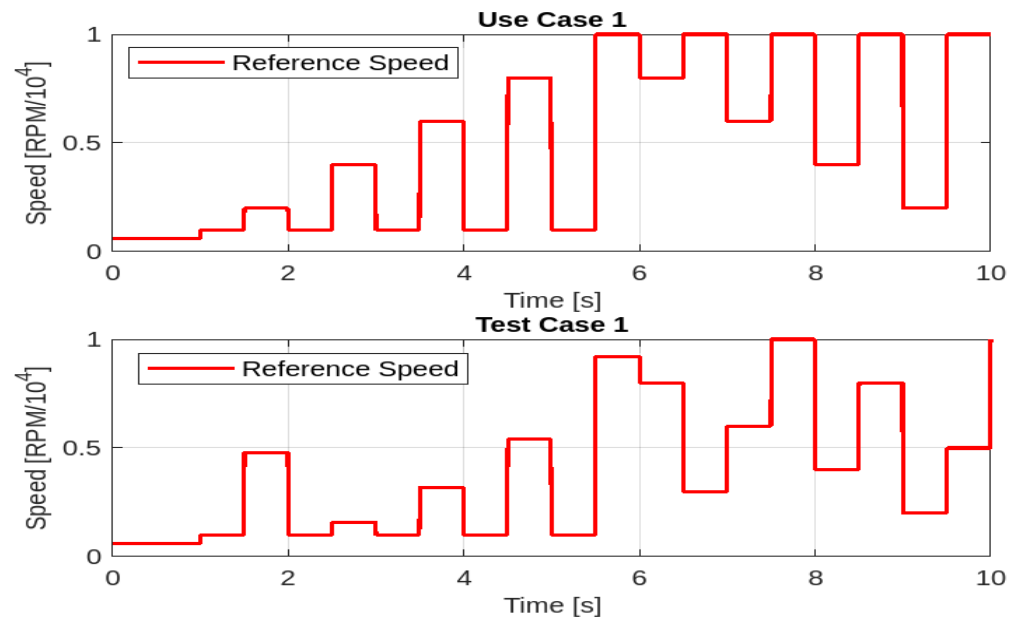


Figure 4. Step Signal Sequence.

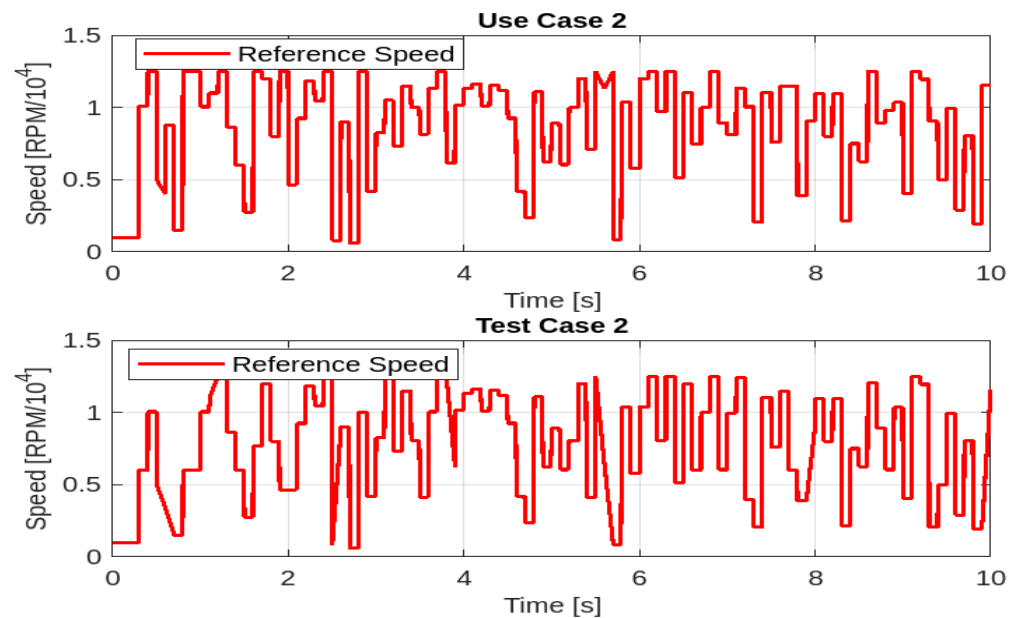


Figure 5. Combined Step and Ramp Signal.

4.1. Use Case 1: Step Signals

The first use case shown in Figure 4 employs a sequence of step signals with varying amplitudes [26], representing sudden reference speed changes. The step function, defined as

$$f(x) = \sum_{i=0}^n \alpha_i \chi_{A_i}(x), \quad \text{for all real numbers } x \quad (11)$$

where α_i are the step amplitudes, A_i are disjoint intervals, and $\chi_{A_i}(x)$ is an indicator function, tests the controller's response to abrupt speed transitions. This signal challenges the controller by introducing an average of two speed transitions per second [23], each with varying amplitudes (α_i), exposing its robustness limitations:

$$\chi_{A_i}(x) = \begin{cases} 1 & \text{if } x \in A_i \\ 0 & \text{if } x \notin A_i \end{cases} \quad (12)$$

The intervals A_i are designed to avoid overlap, and their union spans the domain of the test. These step transitions test the controller's ability to manage rapid speed changes.

4.2. Use Case 2: Combined Step and Ramp Signals

The second use case represented in Figure 5 combines step and ramp signals to evaluate the controller's ability to handle both abrupt and gradual speed variations. Here, the ramp function is defined as shown in Equation (13):

$$R(x) := x \cdot [x \geq 0] \quad (13)$$

To enhance its applicability, a modified ramp equation adjusts the starting time t_0 and slope m :

$$R(t) = m \cdot (t - t_0) \cdot \chi_{[t_0, \infty)}(t) \quad (14)$$

This configuration creates linear speed changes that test the controller's capability to manage gradual accelerations or decelerations. Designed to initiate an average of 10 transitions per second, this signal introduces extreme challenges, particularly in high-transition-rate scenarios.

By combining step and ramp signals [26], this use case simulates real-world conditions where motors experience sudden speed shifts alongside gradual changes. The concatenated signals expose the PI controller's limitations in scenarios demanding rapid transitions and high responsiveness.

5. Devised Solution

As mentioned in Section 1, the approach proposed in this paper is one which utilizes the benefits of tiny NNs (reduced memory footprint and computational complexity) by inserting them in the FOC loop to enhance the performance of conventional PI controllers during the control sequence. Two scenarios are developed in this section, showcasing the ability to unveil and learn the hidden dependencies in the control signals, and properly readjust the reference or desired signals. With high frequencies of speed transition, the PI controller is certain to generate suboptimal responses, which need to be corrected. This corrective approach can be made by integrating the TinyFC in either the speed control unit or the current control unit. In this paper, both approaches are explored, and the results are discussed. In the speed control unit, the idea is to improve the PI control operation generating the reference quadrature current i_q . As mentioned in Section 3, this PI controller in the speed control unit compares the reference speed with the measured speed at each time step, predicting the reference quadrature current i_q . This reference current is then used to adjust the voltage supplied to the PMSM, aiming to bring the actual quadrature current in line with the predicted value. However, as shown in Figure 6, due to the limitations of the PI controller during transient periods, there are often significant deviations between the proposed reference current and the actual current.

The same behavior can be observed in the current control unit. The current control unit contains two PI controllers, one which generates the direct voltage v_d and the other the quadrature voltage v_q . Since the direct and quadrature attributes of the PMSM (voltage and current) are not independent, it is hence sufficient (from equation PMSM currents) to control just one of these attributes. For this study, the current control unit explored is the quadrature current control unit, which generates the quadrature voltage. This is due to the initial condition, which is to set the direct current to zero, which makes it more complex to determine the shape of the ideal direct voltage. However, it is possible to determine the ideal quadrature voltage mathematically using approximation methods, which are detailed in Section 5.1. Similarly to the speed control unit, the quadrature current control unit also

performs a signal comparison to generate the quadrature voltage which will be used to control the PMSM. The PI in the quadrature current control unit compares the reference quadrature current generated in the speed control unit to the measured quadrature current, and generates the quadrature voltage which should be used to readjust the speed of the PMSM. The limited control ability of the PI controller in this case can be observed in the shape of the quadrature voltage, with spikes leading to a poor control. To address this issue, a TinyFC model is introduced as a corrective mechanism.

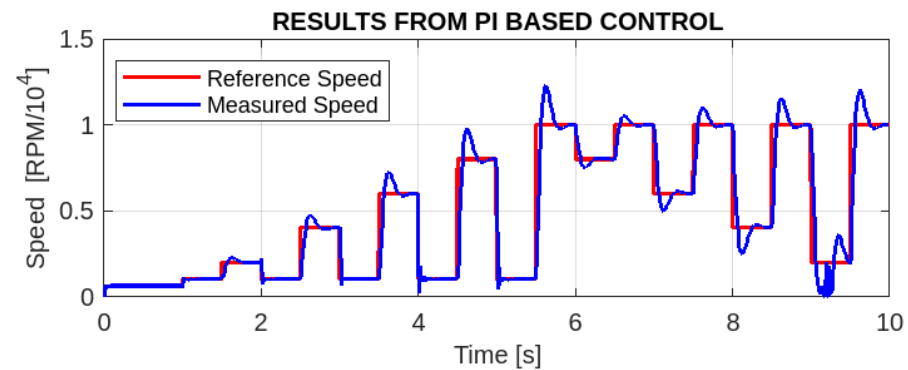


Figure 6. Observation of PI-based control (Use Case 1).

The TinyFC model is designed to capture the underlying dependencies between the input data listed in Table 2, namely, the reference speed, the measured speed, and the proposed PI generated quadrature current i_q when inserted in the speed control unit; in the case of the quadrature current control unit, these are the reference quadrature current, measured quadrature current and the PI generated quadrature voltage. By analyzing these inputs, the TinyFC learns to detect deviations in the PI-prediction. The model continuously assesses how much the proposed values deviate from the desired trajectory, and through a addition block modeled in Simulink, it progressively corrects or readjusts the values generated by the PI controller. The adjusted signal can hence be defined as in Equation (15):

$$x^{adj} = x^{ref(PI)} + \Delta x^{TinyFC} \quad (15)$$

where $x^{ref(PI)}$ is the PI prediction (depending on the unit in which the TinyFC is inserted), Δx^{TinyFC} is the corrective value by which the PI prediction is adjusted, and x^{adj} is the adjusted signal, which enables a smoother control over the simulation time.

Table 2. Parameters composing the dataset used in the training of the TinyFC model.

Unit	Notation	Parameter	I/O
Speed Control	ω^{ref}	Reference Speed	Input
	ω^{meas}	Measured Speed	Input
	i_q^{PI}	PI-predicted Quadrature Current	Input
	Δi_q^{TinyFC}	Current Compensation	Output
Current Control	i_q^{PI}	PI-predicted Quadrature Current	Input
	i_q^{meas}	Measured Quadrature Voltage	Input
	v_q^{PI}	PI-predicted Quadrature Voltage	Input
	Δv_q^{TinyFC}	Voltage Compensation	Output

5.1. Dataset and Pre-Processing

This work utilizes a supervised learning method, requiring the TinyFC to be trained with reliable ground truth values. The dataset used for training is generated based on the

reference error, which is a measure of the deviation of the observed values with respect to the ideal or desired values over the entire simulation period. One of the challenges here is that the PI controller may take time to stabilize, or in some cases, may not stabilize at all. This makes it crucial to carefully define the error signal (vector) used for training the TinyFC, ensuring it accurately reflects the system's behavior. The following subsections define the different pre-processing methods utilized to obtain a proper ground truth, for both scenarios (TinyFC in the Speed control unit and in the current control unit), with the current control unit providing a more straightforward approach.

5.1.1. Speed Control Unit TinyFC

To obtain an accurate ground-truth, the reference quadrature current and the measured speed are observed during the PI-based FOC simulation. Since both signals share the same sampling time, it is possible to identify areas in which the quadrature current exceed the desired interval by comparing its evolution over time to that of the measured speed. Based on the characteristics of each use case, specific data collection strategies are employed to ensure that the TinyFC learns from accurate and representative examples of system behavior.

In the first use case displayed in Figure 6, considering a control interval, it is observed that the PI oscillates before stabilizing. These oscillations make the predicted current by the PI controller increase. This can be seen on the reference quadrature current scope, which also oscillates abnormally before settling. Since both the speeds and the reference quadrature current have the same sampling time (T_s), it is safe to assume that the range in which the measured speed follows the reference speed corresponds to the range in which the reference quadrature current generated by the PI controller is within an acceptable threshold T . Thus, the threshold acts as a filter that limits or caps values that overshoot. Essentially, when the system's response (whether in terms of speed or current) exceeds this threshold, the system restricts the values to prevent overshooting. This filtering process ensures that the signals remain within a controlled range. In this context, T could also be represented as a limiter function that prevents undesirable spikes or overshoots in the reference quadrature current i_q and measured speed ω^{meas} . Thus, the filtering threshold for the given signal $x(t)$, for both speed $\omega(t)$ and quadrature current $i_q(t)$, is defined according to Equation (16):

$$x_{filtered}(t) = \begin{cases} x(t), & \text{if } |x(t)| \leq T \\ T, & \text{if } x(t) > T \\ -T, & \text{if } x(t) < -T \end{cases} \quad (16)$$

If $x(t)$ exceeds T in magnitude, the filter caps the signal at T (or $-T$ for undershoots) as illustrated in Figure 7.

The filtered reference i_q is thus used as the desired quadrature current, and thus subtracted from the PI quadrature current vector after simulation, to obtain the ground truth for the first case. This ground truth determines by which values the PI predictions should be corrected at each time step.

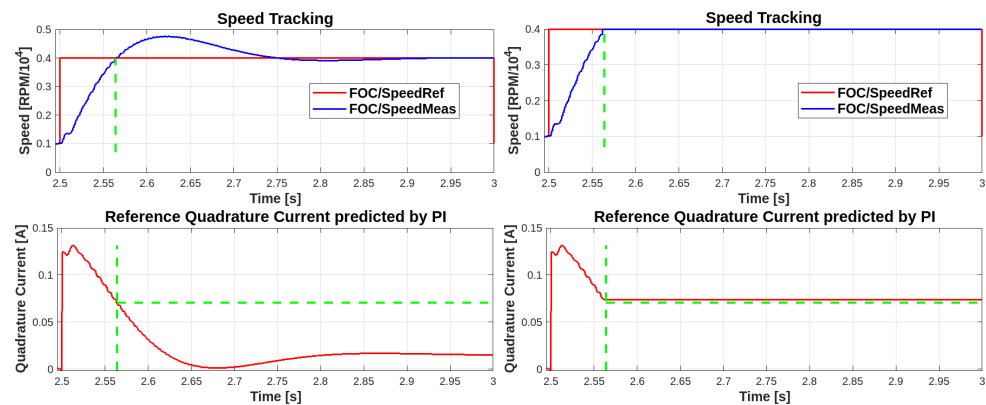


Figure 7. Quadrature current from PI-controller simulation adjusted using speed observations threshold.

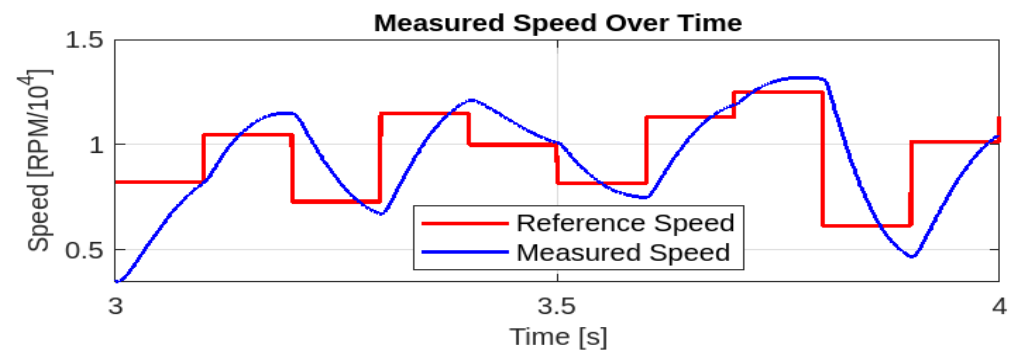


Figure 8. Observation of PI-based control (Use Case 2).

Regarding the second use case shown in Figure 8, the PI controller's response demonstrates a significant instability, resulting in an inability to track the reference speed consistently. This behavior renders it impractical to apply the same correction methodology used in the first case, where overshoot mitigation was possible with a more stable response profile. Given the increase in transitions as compared to the first use case and the frequent oscillations in the PI controller's output, there are very few time intervals where the measured speed adequately tracks the reference speed trajectory. In such cases, where the system does not exhibit reliable steady-state behavior, the approach shifts toward capping the initial and final values within the range where the reference speed closely follows the measured speed, then transforming these capped values using an ideal signal response function.

The method proposed involves a variant of exponential decay and growth to model the desired system response. Exponential decay is commonly used to model processes where a quantity decreases towards a steady-state value over time. Mathematically, the standard form of an exponential decay function is given by Equation (17):

$$y(t) = y_0 e^{-\frac{t}{\tau}} \quad (17)$$

where we have the following:

- $y(t)$ represents the value of the signal at time t ;
- y_0 is the initial value of the signal at $t = 0$;
- τ is the decay constant, which determines the rate of decay.

Most often for control purposes, a more sophisticated signal transformation is needed to account for transitions between initial and final states. The modified decay function,

which models the system's transition towards a final set-point over time, is expressed as shown in Equation (18):

$$y(t) = y_{\text{final}} + (y_{\text{initial}} - y_{\text{final}}) \cdot \left(1 - e^{-\frac{t}{\tau}}\right) \quad (18)$$

This modified form smoothly transitions the signal from an initial value y_{initial} to a final value y_{final} , where the exponential term ensures a gradual approach to the final value over time. This equation mimics the ideal system response as the controlled variable reaches a desired set-point as shown in Figure 9.

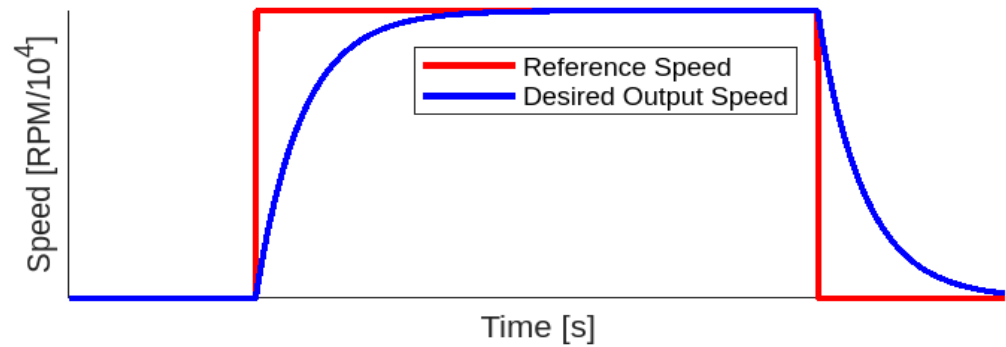


Figure 9. Exponential signal curve.

However, in the context of the second use case, certain intervals of the measured signal do not fully represent the possible range of values that the system can take due to the abrupt transitions and noise in the data. Applying a steep approach toward the final value as described in the modified decay function can over-constrain the model by forcing it to settle too quickly, especially in cases with limited representative transitions. To address this, the decay function is adjusted to allow more flexibility and a more gradual approach towards the final value, reducing the constraints on the model and allowing it to learn from a broader range of transitions. The readjusted function is shown in Equation (19):

$$y(t) = y_{\text{final}} + (y_{\text{initial}} - y_{\text{final}}) \cdot e^{-\frac{t}{\tau}} \quad (19)$$

This modification reduces the aggressiveness of the approach towards y_{final} , thereby allowing the system to follow a more natural and less constrained path, particularly useful in complex, rapidly changing signals. The decay constant τ is crucial in this equation, as it governs the slope of the function, determining how quickly the signal transitions from its initial value to the final desired value. A smaller τ results in a faster transition, while a larger τ allows for a more gradual change.

Similar to the speed signal, the reference quadrature current can be capped and modified using the adjusted exponential decay function. By observing the reference speed and measured speed over time (which share the same sampling time T_s), the reference quadrature current is adjusted at each transition using Equation (20), derived from Equation (19):

$$i_q^{\text{ref}}(t) = i_q^{\text{final}} + (i_q^{\text{initial}} - i_q^{\text{final}}) \cdot e^{-\frac{t}{\tau}} \quad (20)$$

This function ensures that the reference quadrature current transitions smoothly between its initial and final values over the simulation time. The decay constant τ can be tuned to adjust the rate of change, ensuring that the current values do not deviate too rapidly, preventing the continuous oscillations and overshoot commonly associated with PI controllers in high-frequency changing states. Figure 10 shows the capping and rectification applied to the observed quadrature current over a sample interval.

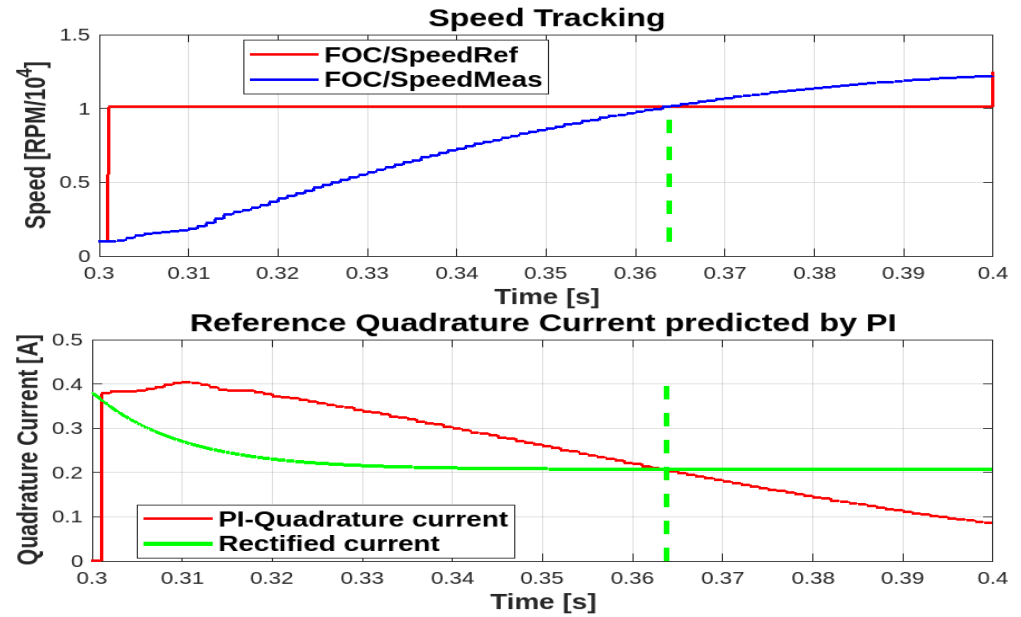


Figure 10. Quadrature current adjusted based on capping and rectification of the measured speed.

5.1.2. Current Control Unit TinyFC

The error vector (ground-truth) used to train the TinyFC for the quadrature current control loop can be obtained in two different ways.

Ideally, the direct current is set to zero, which enables to automate and facilitate measuring the ideal quadrature voltage as compared to the signal capping introduced in Section 5.1.1. Since the reference speed is given, and the direct current is set to zero, it is thus sufficient to approximate the PMSM as a constrained dynamic system using a fourth-order Runge–Kutta approximation [27,28] or an *fmincon* optimization technique [29] to predict the future voltages of the controlled system. From Equation (4), the voltage equations can be obtained. Though both methods are explored, a focus on *fmincon* is made, which considers constraints and feedback to optimize its prediction. The control objective is set to the following:

1. Track the reference speed ω^{ref} , thereby determining the desired electrical angular speed.
2. Ensure that the currents i_d and i_q satisfy the operational constraints:
3. i_d is typically set to zero for MTPA, while i_q is adjusted to control torque.

With respect to the voltages, the dynamic equations of the motor could be redefined as listed in Equation (21):

$$\begin{aligned} v_d &= R_s i_d + L_d \frac{di_d}{dt} - \omega_e L_q i_q \\ v_q &= R_s i_q + L_q \frac{di_q}{dt} + \omega_e L_d i_d + \omega_e \lambda_{pm}. \end{aligned} \quad (21)$$

This optimization approach is typical to MPC, used to ensure that the voltages v_d and v_q are calculated iteratively at each time step, while respecting the current and voltage constraints. The control problem as formulated in Equation (22) involves determining the optimal v_d and v_q that minimize the error between the measured mechanical speed ω^{meas} and the reference one ω^{ref} , subject to constraints on voltages, currents, and motor parameters:

$$g(v_d, v_q) = \left(\omega_{(v_d, v_q)} - \omega^{\text{ref}} \right)^2, \quad (22)$$

The updated voltages are used to recompute i_d and i_q , which in turn are used to recompute the speed ω^{meas} (from the updated currents). The computed speed compared with ω_m^{ref} is used to adjust the control inputs for the next step. The optimization problem to

solve at each time step is formulated as shown in Equation (refoptimeq) with the constraints being relative to the PMSM characteristics listed in Section 3:

$$\min_{v_d, v_q} g(v_d, v_q) = \left(\omega(v_d, v_q) - \omega^{\text{ref}} \right)^2 \quad (23)$$

The estimated voltage vector v_q is considered the ideal voltage from which the deviation of the PI predicted v_q is subtracted. This error vector is used similarly as in Section 5.1.1.

A corrective approach similar to that which was applied in the speed control unit can be used to obtain the expected quadrature voltages and hence the error vector, by subtracting the PI prediction from it. In this case, the corrective measure only ensures the predicted values are in range, rather than forcing the signal to follow an alternative path dictated by the optimization problem solved by the *fmincon* algorithm. Thus, the results from this corrective approach are shown in Section 7.1, with further remarks on both approaches reported in Section 7.3.

As illustrated in Figure 11, the quadrature voltage increases proportionally with an increase in speed. With similar profiles, it becomes trivial to determine the intervals which create overshoots in the control sequence. These intervals are therefore handled similarly as in Section 5.1.1.

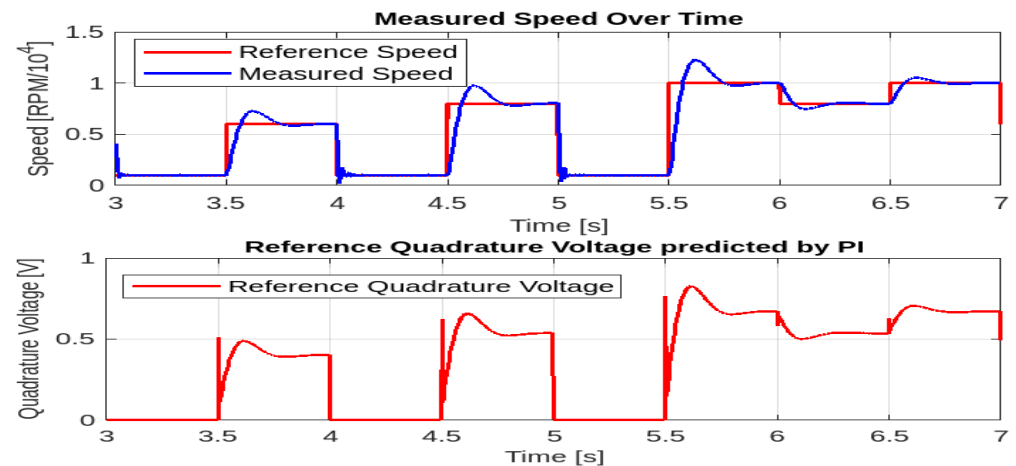


Figure 11. Correlation between the speed and quadrature voltage profiles over a sample interval from the control sequence.

5.2. TinyFC Design

The proposed NN architecture illustrated in Figure 12 features 1400 parameters NN with a distinctive structure, combining two main branches of five fully connected (FC) layers, and two residual layers per branch. The input layer receives the raw input features, represented as x . This layer does not transform the data but simply serves as the input point of the NN. From this layer, the NN branches into two main parallel streams right after the input layer. Each branch processes the input independently (with a different number of neurons per layer) through multiple layers, which allow the NN to learn different representations of the data, with the streams re-merging later in the network. Each sub-branch follows a similar pattern, which is expressed by defining each layer's function within each branch. Each FC layer performs a linear transformation of the input data, with the output z of the FC layer represented as formulated in Equation (24):

$$z = Wx + b \quad (24)$$

where W is the weight matrix, and b is the bias vector. Complex patterns in the data are thus learned by adjusting W and b during training. Dropout layers inserted after each

FC layer in the main branches randomly set a fraction of their input units to zero during training, effectively regularizing the network and preventing over-fitting.

The outputs of the main branches are added to a residual layers at several instances throughout the network's architecture, allowing an identity mapping from one input point to the output, essentially bypassing one or more layers in the network, which is crucial to address vanishing gradient problems. The combination of the mainstream input and the residual input is performed element-wise. This encourages the merging of different feature representations learned by separate branches, allowing the network to integrate information in a flexible way. Each branch has a Tanh activation layer after the addition layers, applying a nonlinear transformation. The Tanh activation function maps input values to a range between -1 and 1 , which enable gradient flow, allowing the model to approximate more complex functions.

The final FC layer aggregates all processed features and maps them to the final output of the network, whose topology is shown in Figure 12. This TinyFC architecture proposes an enhanced regressive FC model, which can easily be used to understand how the current i_q and voltage v_q vary with respect to the reference or desired speed ω^{ref} .

Explicitly, the model is trained using two steps repetitively, a forward and a backward pass. In the forward pass, the input sequence (3×1) is passed through the first FC layer of the main streams, with the sequence represented as a vector as described in Equation (25):

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad (25)$$

Since the first FC layers can be considered to have n_1 neurons (32 and 24 neurons respectively), the weight matrix $\mathbf{W}_1$ and bias vector $\mathbf{b}_1$ can be expressed as formulated in Equation (26), with the output of this layer represented in Equation (27):

$$\mathbf{W}_1 \in \mathbb{R}^{n_1 \times 3}, \quad \mathbf{b}_1 \in \mathbb{R}^{n_1 \times 1} \quad (26)$$

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \quad (27)$$

A fraction of the output of this first FC is set to zero via the dropout operation, and sent to the second FC. The second layer (FC) performs a similar operation to the first one, this time with half the number of neurons (as compared to the first FC of each mainstream). Its output is also subjected to a similar dropout (0.1) as in the first layer, and forwards the third FC which has half the number of neurons of the second FC.

The output of this layer is combined with that of a skip connected layer with the same number of neurons, via the addition operation described in Equation (28), where $\mathbf{s}_1$ is the skip connection tensor to be added:

$$\mathbf{a}_2 = \mathbf{z}_2 + \mathbf{s}_1 \quad (28)$$

The vector $\mathbf{a}_2$ obtained goes through a Tanh activation layer, which helps model non-linear relationships in the data. The process is thus repeated with fewer layers downstream as the number of neurons per layer is continuously halved.

During the back-propagation step, the gradient of the loss L described in Section 7.1 is obtained alongside that of the activation function, and fully connected layers $\mathbf{W}_k$ to update the weights, using gradient descent as formulated in Equation (29), where η is the learning rate:

$$\mathbf{W}_k \leftarrow \mathbf{W}_k - \eta \frac{\partial L}{\partial \mathbf{W}_k}, \quad \mathbf{b}_k \leftarrow \mathbf{b}_k - \eta \frac{\partial L}{\partial \mathbf{b}_k} \quad (29)$$

This process is repeated for each layer in the network, propagating the gradients backward and updating the weights and biases.

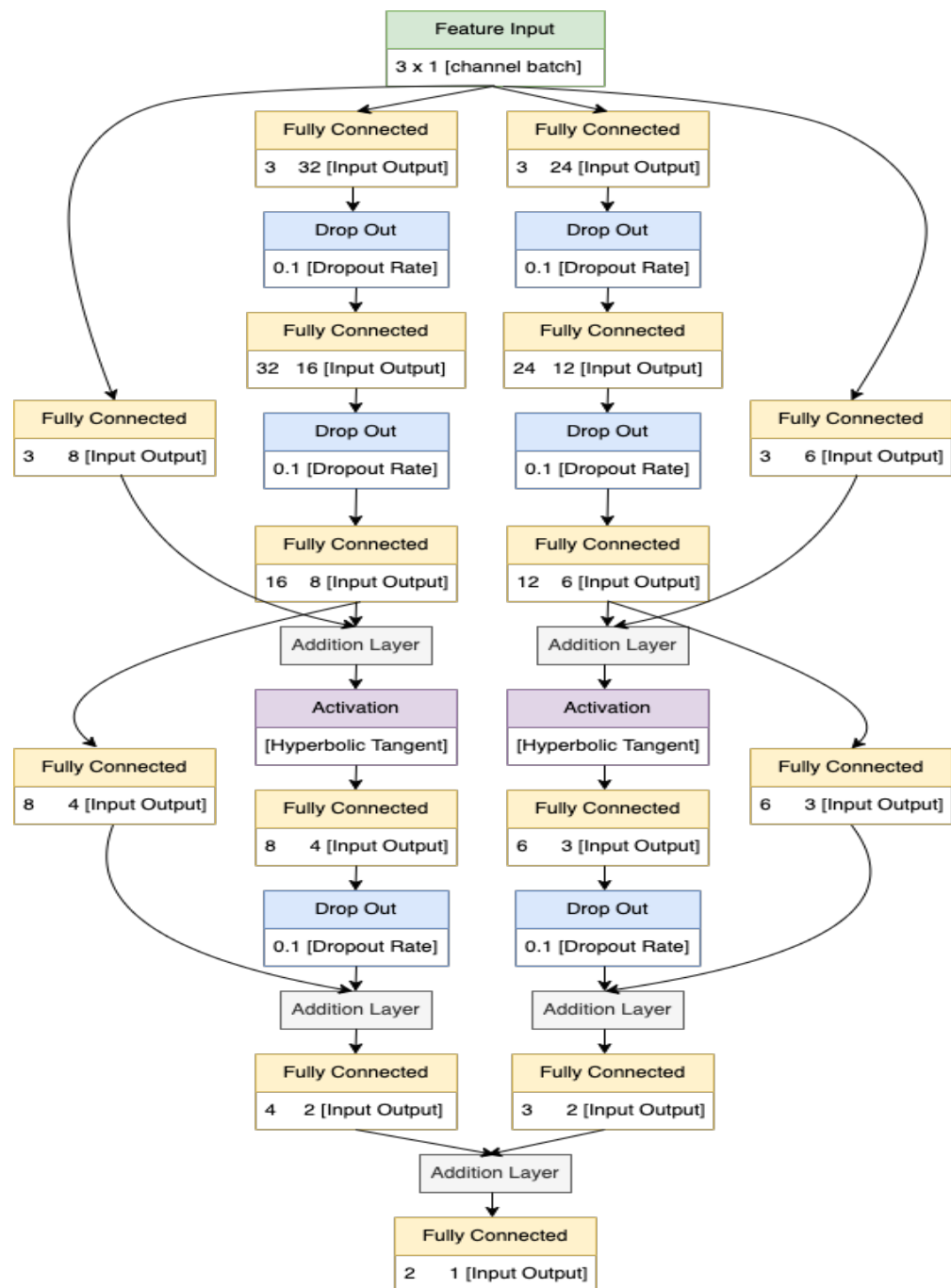


Figure 12. TinyFC architecture.

5.3. Integration of TinyFC in the FOC Loop

The training data comprise a $300,001 \times 4$ sequence per test case, acquired through the Simulink PI-based FOC model. Each 10-second test case is sampled at $T_s = 3.3333 \times 10^{-5}$, corresponding to the PWM switching frequency, yielding $\sim 300,000$ samples. TinyFC is trained using data from test case 1, Section 4.1, and fine-tuned on data from test case 2, Section 4.2. For training, the dataset is split into a training, validation, and test set with the ratio 8:1:1, to validate the model on unseen data. Once trained, the TinyFC is integrated into the FOC, and its corrective performance is monitored from how the measured speed

follows the desired speed signal. The devised and trained model is tested in both the speed and current control units one at a time.

The model is integrated in the control system setup to correct the PI prediction. As shown in Figure 13 (case of integrating the TinyFC in the speed control unit), the model operates in real-time and updates its correction at every sample interval T_s . This ensures that the corrective action is synchronized with the PI controller's output. The reference and measured speed are monitored during this phase, crucial in evaluating the performance of the corrective TinyFC.

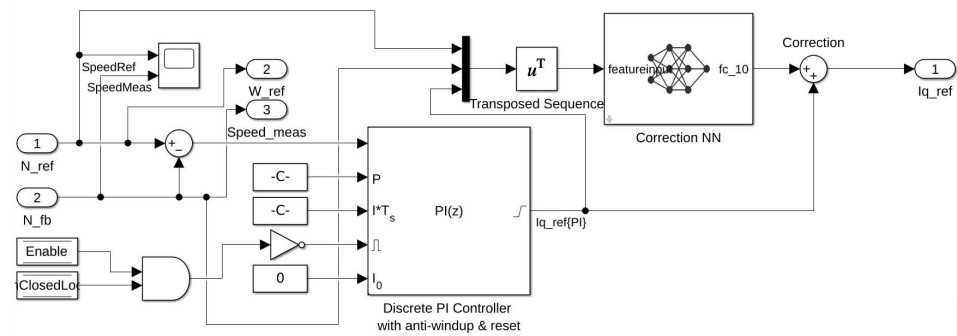


Figure 13. TinyFC in speed control unit subsystem in Simulink.

6. Model Optimization and Compression

A key challenge to the practical implementation of NN-augmented controls is the deployability to devices with limited resources. This work focuses on MCUs widely employed in FOC applications for PMSMs such as the SR5R1-EVBE3000D and the NUCLEO-G474RE boards from STMicroelectronics [30]. Constraints in terms of memory, computational power, and operating frequency present significant obstacles. Therefore, optimizing and compressing models like TinyFC is essential to ensuring the efficient deployment and operation of the applications.

The feasibility of deploying the augmented control system on these resource-constrained MCUs is determined by three critical parameters: the model size of the TinyFC model, its RAM usage during computation, and its inference latency. Increasing the model's complexity by adding more parameters may improve predictive accuracy, but it risks exceeding the memory and processing capabilities of these devices, making real-time implementation impractical. On the other hand, overly simplified models, while resource efficient, may not achieve the accuracy required for effective control loop integration. Consequently, this work focuses on an iterative approach to trade off between model complexity and predictive accuracy. By optimizing hyperparameters (HPs), compressing the model size, and reducing the memory footprint through quantization, we ensure that the model can operate effectively within the limitations of the hardware. These strategies collectively enable the practical deployment of advanced control algorithms, maximizing performance without compromising the constraints of the memory and processing capabilities.

6.1. Hyperparameters Optimization

HPs govern model performance. For TinyFC, the learning dynamics, such as layer size, dropout rate, and learning rate, impact the model accuracy. Hyperparameter tuning (HPT) aims to minimize loss, maximize accuracy, or optimize other metrics like prediction error or F1 score. The computational complexity of manual tuning in high-dimensional parameter spaces is high. Automated HPO techniques, such as Bayesian Optimization (BO), offer efficient solutions.

This research applies BO to identify optimal HPs for the devised TinyFC in the speed control problem, aiming to minimize prediction error. The process begins with selecting parameter ranges, i.e., neuron count, dropout rates, and learning rates. The candidate configurations are ranked based on their performance on the test set [31]. BO uses a probabilistic surrogate model [32], which approximates the objective function $f(\mathbf{x})$ using a Gaussian Process (GP). The GP is defined by a mean function $\mu(\mathbf{x}; \theta)$ and a covariance kernel $k(\mathbf{x}, \mathbf{x}'; \theta)$, where θ represents the kernel parameters. Based on prior evaluations, the GP models the posterior distribution of $f(\mathbf{x})$, enabling informed exploration.

BO utilizes an acquisition function to guide the search for optimal HPs by balancing exploration in high-uncertainty regions and exploitation near the current best result [33]. This work employs the Expected Improvement (EI) acquisition function, formulated in Equation (30):

$$\text{EI}(\mathbf{x}, Q) = \mathbb{E}_Q [\max(0, \mu_Q(\mathbf{x}_{\text{best}}) - f(\mathbf{x}))] \quad (30)$$

where $\mathbf{x}_{\text{best}}$ represents the current best configuration.

In MATLAB version 2024b, BO was implemented using the *bayesopt* function, with HPs specified with the *optimizableVariable* function, defining ranges for the TinyFC parameters such as number of neurons per layer, training iterations, and dropout rates. The objective function is evaluated from training the models and computing the mean squared error (MSE) on a test dataset.

Figure 14 illustrates one such optimization process, with the optimized NN subsequently validated on a test dataset to confirm its effectiveness in minimizing prediction errors [34]. The HP search begins with a random model with the number of parameters defined within the optimization range. This devised model is trained, evaluated, and tested over the split dataset, with the test set's MSE used as the objective to minimize. The process is repeated several times until one of the stopping criteria is met (maximum running time or optimization objective reached). The aim of the continuous search is to progressively converge towards a model with the smallest MSE, thereby solving a convex optimization problem, making use of EI as described earlier in this section.

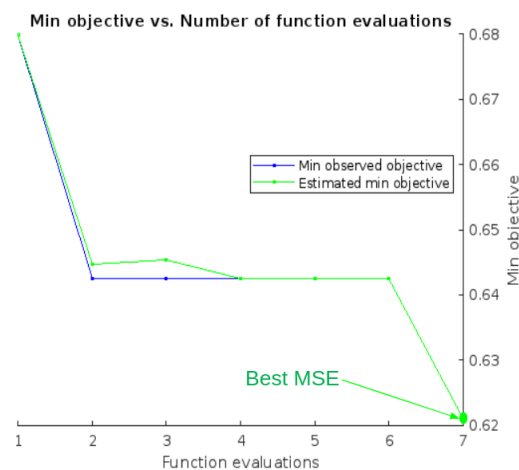


Figure 14. Observed objective function and estimated minimal objective of BO.

6.2. Projection-Based Pruning

Pruning involves removing parameters from the trained model, either individually or in groups such as neurons, to enhance computational efficiency while maintaining accuracy. This technique can be integrated with HPO or applied independently. In this work, pruning is employed independently, with an explanation provided in Section 7.3.

To optimize the TinyFC model, projection-based pruning is utilized. This technique leverages Principal Component Analysis (PCA) to identify and remove redundant neurons within a layer, significantly reducing the TinyFC size. By analyzing neuron activation patterns, PCA-based pruning projects activations into a lower-dimensional space, retaining only neurons that contribute unique variance [35].

Projection-based pruning, also referred to as PCA-based neuron pruning, operates by analyzing variance in neuron activations during a forward pass. The activation outputs form a data matrix X , where rows represent data samples, and columns represent neuron activations. These activations are standardized to form a matrix X_s with zero mean and unit variance. PCA is applied to X_s to derive principal components (PCs) that explain the most variance. The top l PCs are selected as in Equation (31):

$$\sum_{i=1}^l \lambda_i \geq \eta \sum_{i=1}^k \lambda_i \quad (31)$$

where λ_i is the variance explained by the i -th component, k is the total number of neurons, and η (typically 0.8–0.85) defines the variance retention threshold.

The pruning step solves an eigenvalue problem on the covariance matrix $C = X_s^T X_s$ of standardized activations [36]. Eigenvectors corresponding to the largest eigenvalues define directions of maximum variance [37]. Figure 15 illustrates how projection applies to a fully connected layer with two neurons, assuming no bias term.

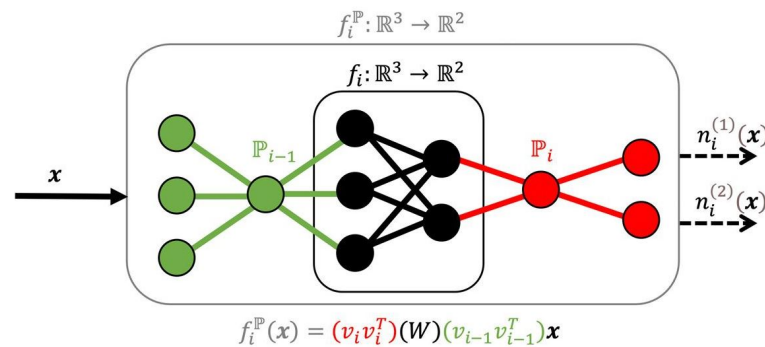


Figure 15. Example of a fully connected layer with rank-1 input and output projection operation. Reprinted with permission from Ref. [38].

Neurons most strongly correlated with these retained PCs are preserved, as they represent the unique variance in the data. Redundant neurons, contributing little additional information, are pruned [39,40]. Fine-tuning the pruned model allows the remaining neurons to adapt and recover any performance loss. This step is essential for maintaining or enhancing predictive accuracy after parameter reduction.

6.3. Quantization

The trained, optimized, and pruned TinyFC models all use single-precision floating-point data types, which require adequate memory and hardware capable of performing 32-bit floating point operations. These requirements are demanding for low-power devices. Quantization addresses this challenge by representing the values with smaller containers, such as 8-bit, and computing with fixed-point operators. This step can reduce both memory consumption and computational complexity [41,42].

Two primary methods are used for quantizing NNs: Quantization Aware Training (QAT) and Post Training Quantization (PTQ) [43]. In QAT, quantization is incorporated by the training process itself [44], while in PTQ, the trained model is quantized after training is completed and without any further fine-tuning. Although QAT is known to result in

better accuracy than PTQ, PTQ is less demanding and offers a simpler workflow since the weights and activations do not change [45]. Therefore, PTQ is used for this research work.

During the PTQ step, the dynamic ranges of the parameters and activations in a network are collected. Exercise the network with sample data that are representative of the training data. Extract the minimum and maximum values of the weights and biases in the fully connected layer. Extract the minimum and maximum values of the activations in all other layers as illustrated in Figure 16.

Scaled 8-bit integer data types have limited precision and range when compared to single-precision floating point data types. Quantization of the weights, biases, and activations of layers can introduce precision loss, underflow, and overflow. In Figure 16, the grey boxes for the activation of the Relu operation show the potential overflow and underflow.

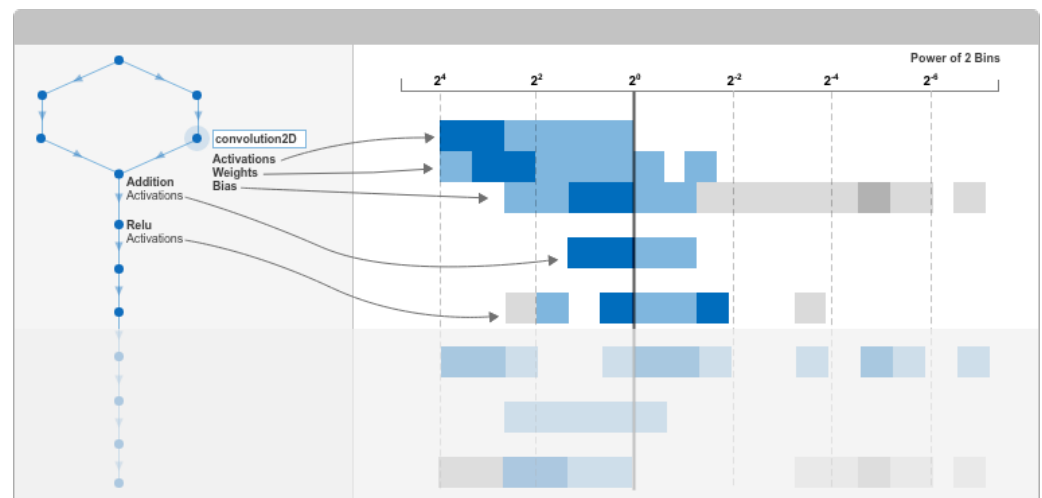


Figure 16. Post-training quantization of network parameters and activation illustration.

The TinyFC model is quantized using the ST Edge AI Developer Cloud's Quantizer [46]. ST Edge AI Developer Cloud leverages ONNX Runtime PTQ to convert the weights and activations of the model in single precision (exported from MATLAB) into 8-bit integer values. During this process, operations are performed using integer representations of tensors, greatly improving the execution speed on MCUs, while maintaining reasonable accuracy. In this case, uniform PTQ is employed, where floating-point weights and activations are mapped to an 8-bit integer range. Quantization is also available in MATLAB using the *deepNetworkQuantizer* [37] with similar numerical results.

7. Experimental Results

The original PI-based FOC and the TinyFC augmented FOCs proposed by this work are compared across multiple evaluation criteria. These criteria serve as key performance metrics to assess the effectiveness of the proposed augmentations in enhancing the overall control loop. This section introduces the metrics, provides a comparative analysis of the performance, and interprets the results obtained throughout the workflow described in Section 5.

7.1. Control Loop Evaluation Metrics

The primary evaluation metric employed for training the TinyFC in this study is the MSE, a widely used indicator of predictive accuracy and model performance in machine learning and statistical modeling. MSE, or MSD, quantifies the average squared discrepancy between the predicted and observed values, thereby measuring the quality of an estimator.

Mathematically, MSE is the expectation of the squared error loss and serves as a risk function, representing how much the model's predictions deviate from the actual value [47].

Due to its derivation from the square of the Euclidean distance, MSE is always a non-negative value, reaching zero only when predictions perfectly align with actual values. This property makes MSE a robust and interpretable metric, with a lower MSE directly correlating with higher prediction accuracy, allowing it to serve as an effective tool for monitoring model convergence during training and providing a clear indication of the model's fit to the data [47]. MSE is computed for a vector of predictions across n data points, where Δi_q^{GT} and Δv_q^{GT} denote the observed values, i.e., the ground truth error vector obtained from the PI deviation (in both control units), and Δi_q^{TinyFC} and Δv_q^{TinyFC} represent the model's predictions. Equations (32) and (33) show the MSE formulation used to train the TinyFC model:

$$MSE = \frac{1}{n} \sum_{i=1}^n \left(\Delta i_q^{GT} - \Delta i_q^{TinyFC} \right)^2 \quad (\text{In the speed control unit}) \quad (32)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n \left(\Delta v_q^{GT} - \Delta v_q^{TinyFC} \right)^2 \quad (\text{In the current control unit}) \quad (33)$$

Model complexity, quantified by the number of learnable parameters, also plays a critical role, as it directly impacts computational efficiency and deployability. The number of non-zero parameters of the TinyFC is monitored, which quantify the complexity or the number of matrix operations (multiply and accumulate or MACC) required to be performed within the model during deployment.

Table 3 reports the evaluation of the MSE for the two considered test cases in both the speed and current control loops.

Table 3. Mean squared error of the trained and optimized TinyFC versions.

Control Loop	Model	Number of Parameters	MSE (%)
Speed	TinyFC	1400	4.09
	HPO TinyFC	670	0.31
	Pruned TinyFC	873	9.19
Current	TinyFC	1400	17.96
	HPO TinyFC	305	0.622
	Pruned TinyFC	814	33.89

In assessing the performance of the augmented FOC in Simulink, improvements are monitored based on the overshoot generated throughout the control sequence and the deviations from the reference control signal. Here, the overshoot is quantified as the extent to which the system's response temporarily exceeds the target setpoint, observable as instances where the measured speed surpasses the reference speed. Specifically, the maximum overshoot is defined as the highest peak in the response curve relative to the system's intended steady-state response [26]. For a given use case, the overshoot formulation can be simplified as shown in Equation (34), where ω_{max}^{meas} is the maximum measured speed, and ω_{top}^{ref} and ω_{base}^{ref} are the highest and lowest reference speeds respectively over a sample interval:

$$Overshoot = \frac{(\omega_{max}^{meas} - \omega_{top}^{ref})}{(\omega_{top}^{ref} - \omega_{base}^{ref})} \times 100 \quad (34)$$

Deviation, on the other hand, refers to the difference between the reference signal and the controller's output signal. This metric provides insight into the control system's ability to accurately track the reference input and the duration required to achieve stabilization:

$$\text{Average Dev} = \frac{1}{n} \sum_{i=1}^n |\omega_i^{\text{ref}} - \omega_i^{\text{meas}}| \quad (\text{Average deviation}) \quad (35)$$

$$\text{Max Dev} = \arg \max |(\omega^{\text{ref}} - \omega^{\text{meas}})| \quad (\text{Maximum deviation}) \quad (36)$$

As expressed in Equations (35) and (36), two primary deviations are monitored: average deviation and maximum deviation. Average deviation reflects the overall performance of the control unit in maintaining consistent tracking of the reference signal. Maximum deviation highlights regions where the control system encounters specific challenges. Table 4 shows a performance assessment between the originally devised PI-based FOC and the different augmented controls (both in the speed and current control units).

Table 4. Performance comparison between PI and the different TinyFC models in terms of max and average deviation and max overshoot.

Control Unit	Test Case	Controller Setting	Max Deviation	Average Deviation	Max Overshoot
Speed	1	PI	0.81	0.05	0.24
		PI + TinyFC	0.89	0.02	0.03
		PI + HPO TinyFC	0.896	0.0316	0.06
		PI + Pruned TinyFC	0.9	0.02	≈0
	2	PI	1.21	0.18	0.25
		PI + TinyFC	1.19	0.15	0.08
		PI + HPO TinyFC	1.23	0.15	1.24
		PI + Pruned TinyFC	1.20	0.16	0.03
Current	1	PI	0.81	0.05	0.24
		PI + TinyFC	0.7	0.03	0.11
		PI + HPO TinyFC	0.81	0.05	0.24
		PI + Pruned TinyFC	0.90	0.04	0.2
	2	PI	1.21	0.18	0.25
		PI + TinyFC	1.22	0.16	0.104
		PI + HPO TinyFC	1.21	0.18	0.20
		PI + Pruned TinyFC	1.19	0.18	0.20

7.2. Deployability on the MCU

A critical evaluation criterion for the MCU deployment of TinyFC is the inference time, as it determines the model's viability in low-latency environments. The deployability of the models across various MCUs is analyzed using the ST Edge AI Developer Cloud, which evaluates deployment feasibility and operational efficiency, focusing on key operations such as matrix multiplications (MatMul). Comprehensive analysis of the models' compatibility with MCU resources is conducted by tracking metrics such as MACC, flash memory usage, random access memory (RAM) usage, and execution time. These metrics encapsulate the computational and memory demands critical for successful MCU deployment.

This section explores the implementation of optimized TinyFC models on two widely used MCU platforms (shown in Figure 17) for PMSM control and FOC: the STM NUCLEO-G474RE (for industrial applications) and the SR5E1-EVBE3000D (for automotive applications). The goal is to ensure that the trained and optimized models meet the rigorous memory and processing requirements of FOC applications in real-time.

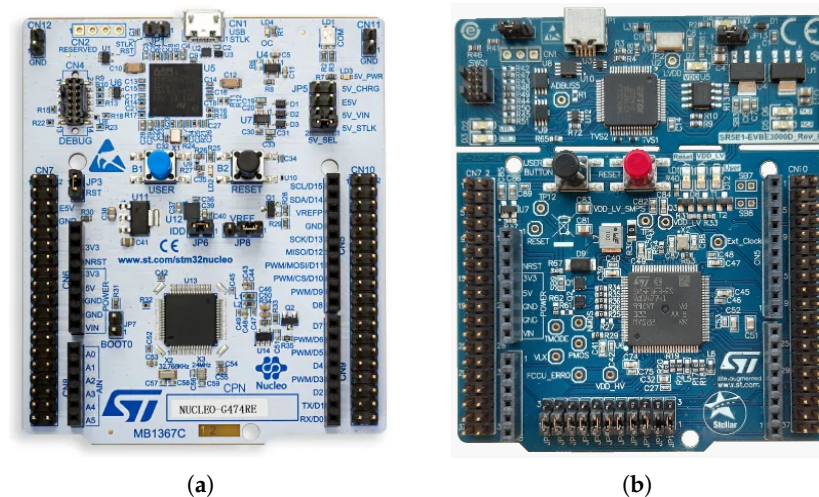


Figure 17. MCU boards used for inference testing. (a) NUCLEO-G474RE (STM32G4); (b) SR5E1-EVBE3000D (StellarE).

The evaluation focuses on the integration of these models into the FOC speed and current control loops, with performance assessed against key parameters:

- **MACC:** Represents the computational complexity, specifically the number of multiplication and accumulation operations required, which directly impacts the processing time.
- **Flash memory usage:** Indicates the storage requirements for the model, critical for maintaining firmware efficiency and accommodating updates.
- **RAM usage:** Reflects dynamic memory requirements during inference, an essential factor to achieve low power operations.
- **Inference time:** Measures the time taken to produce output during real-time operations, where minimal latency is vital for responsive control.

The NUCLEO-G474RE, based on the STM32G4 series with an ARM Cortex-M4 core, operates at a clock speed of 170 MHz. With 512 kilobytes (KiB) of flash memory and 128 KiB of RAM, it is optimized for industrial applications requiring a balance between computational power and energy efficiency, making it suitable for moderately complex NN models and deterministic FOC algorithms [30]. Conversely, the SR5E1-EVBE3000D board, with a clock speed of 300 MHz, 1920 KiB of flash memory, and 256 KiB of RAM, offers higher computational resources and is less constrained, enabling shorter inference times for NN models in automotive applications.

Table 5 presents comparative results for the proposed TinyFC models, including pruned-quantized models evaluated using the ST Edge AI Developer Cloud, highlighting their performance across these key metrics.

Table 5. Neural network characteristics on MCUs.

Control Unit	Model	MACC	FLASH (KiB)	RAM (KiB)	Time (μs)
Speed	TinyFC	1620	Weights: 5.68 Lib STM32: 15 Lib StellarE: 16	Act: 0.199 Lib: 6	G474RE: 207.4 EVBE3000D: 92.8
	HPO	764	Weights: 2.61 Lib STM32: 15 Lib StellarE: 16	Act: 0.152 Lib: 6	G474RE: 144.8 EVBE3000D: 69.4
	Pruned	1034	Weights: 3.28 Lib STM32: 20 Lib StellarE: 20	Act: 0.148 Lib: 10	G474RE: 232.2 EVBE3000D: 102.4
	Quantized	910	Weights: 1.37 Lib STM32: 32 Lib StellarE: 30	Act: 0.383 Lib: 13	G474RE: 371.7
	Pruned				EVBE3000D: 167.9

Table 5. Cont.

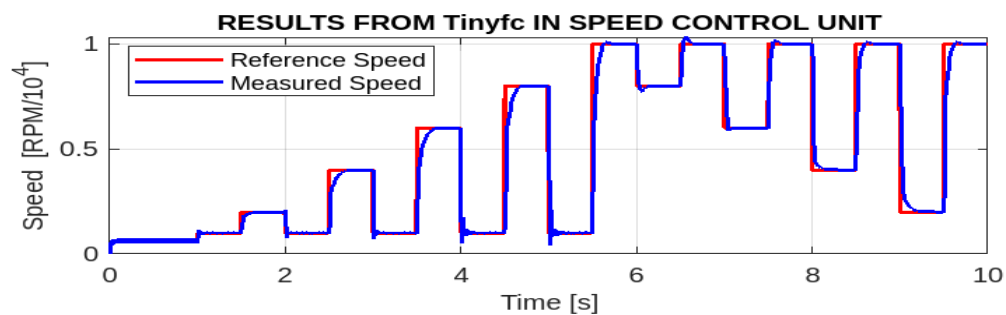
Control Unit	Model	MACC	FLASH (KiB)	RAM (KiB)	Time (μ s)
Current	TinyFC	1620	Weights: 5.68 Lib STM32: 15 Lib StellarE: 16	Act: 0.199 Lib: 6	G474RE: 206.6 EVBE3000D: 92.95
	HPO	431	Weights: 1.18 Lib STM32: 15 Lib StellarE: 18	Act: 0.117 Lib: 6	G474RE: 124.8 EVBE3000D: 61.83
	Pruned	975	Weights: 3.06 Lib STM32: 20 Lib StellarE: 21	Act: 0.148 Lib: 10	G474RE: 229.4 EVBE3000D: 102
	Quantized Pruned	851	Weights: 1.53 Lib STM32: 32 Lib StellarE: 30	Act: 0.296 Lib: 13	G474RE: 365.7 EVBE3000D: 166

7.3. Discussion

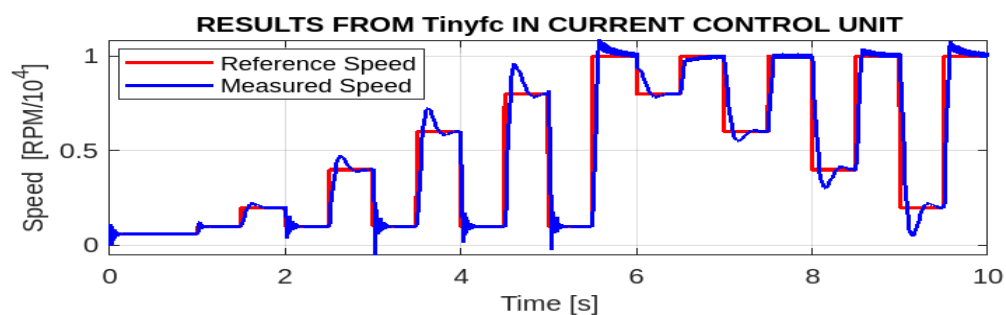
The ground-truth vector used to train the model used in both the speed and current control units is solely derived via the saturation approach involving capping and a threshold mentioned in Section 5.1.1. Though deriving the reference quadrature voltage via state-space equations as mentioned in the first part of Section 5.1.2 will optimize the quadrature voltage predicted, the dissimilarity between the PI predicted quadrature voltage and the quadrature voltage obtained in this case is quite large, which makes it more of an overall rectification at each time step than a correction; hence, this perspective is not presented here. The results presented in Section 7.1 show the performance of the devised and optimized TinyFC models on the test data and both control units. Although inserting the TinyFC model in the current control unit provides an overall reduced deviation and smoother control at low speeds, the controller yet suffers from the presence of overshoots, which worsen when the model is optimized, exposing the flaws of using MSE as a validation metric when training the model. This observation lays the foundation for applying pruning independently to the trained model, regardless of the low MSE observed during the Bayesian Optimization step. It is worth mentioning that the current control unit is not an independent control unit itself since it inherits command of the reference quadrature currents from the speed control unit, making it subject to perturbations induced by this one. In addition to these perturbations, the formulation of the quadrature voltage displayed in Equation (21) clearly highlight the influence of the direct axis currents on the speed quadrature voltage.

On the other hand, the speed control unit provides much better control as shown in Figure 18, with the pruned TinyFC model almost completely removing the presence of overshoots in the control sequence as displayed in Figure 19. Similarly to the current control unit, the TinyFC HPO model does not perform as expected, although results during training, validation, and testing indicate otherwise. Due to noise in the ground truth data and the fact that the training process does not completely cover a feedback comparison of reference and measured speed values, the MSE value, though quite little, cannot fully account for the capability of the model to comprehend or regularize the PI predictions.

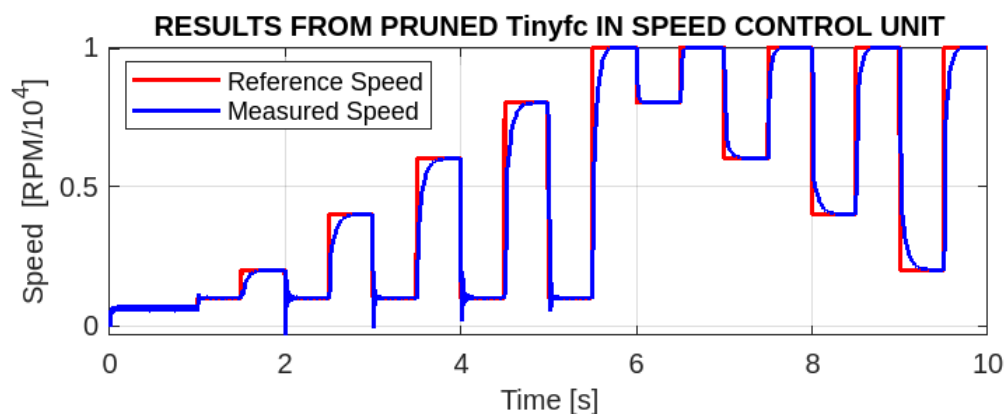
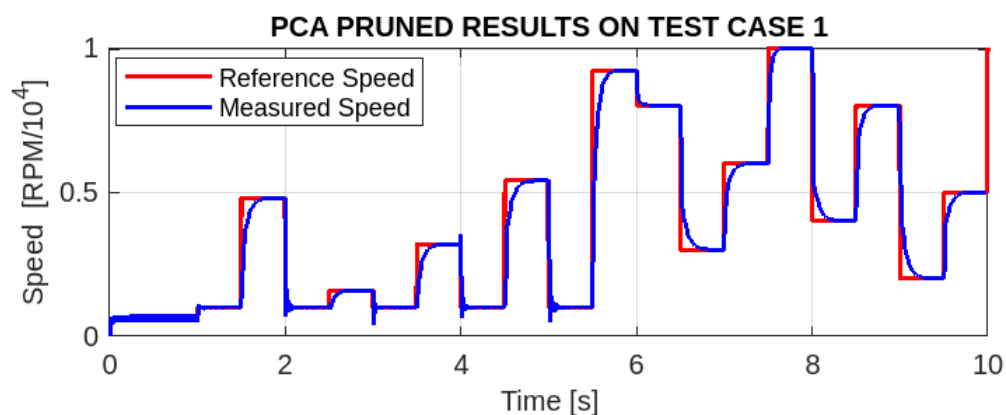
A more robust metric could be the combination of the MSE during training and the evaluation of this error reduction in the PMSM equations. This implies using physics-informed NNs (PINNs) which unlike MSE, provide a more physics-friendly approach [48,49]. In addition, the PCA-pruned TinyFC integrated in the speed control unit which produces the best results is also used on the additional test cases mentioned in Section 4, with the best results on the first test case shown in Figure 20.



(a) TinyFC in Speed Control Unit



(b) TinyFC in Current Control Unit

Figure 18. Results of devised TinyFC in both control units.**Figure 19.** Control loop results obtained by inserting the PCA-pruned TinyFC in the speed control unit.**Figure 20.** Testing generalization ability of PCA-pruned TinyFC (test case 1).

Adding another TinyFC to the direct current control unit enhances the overall corrective performance. As discussed earlier in this section, the interdependence between the direct and quadrature currents complicates the voltage control when focusing only on the quadrature voltage. Using two TinyFC models in the current control unit improves the

sequence as shown in Figure 21. However, this approach increases the complexity of the control loop, making deployment on MCUs more challenging.

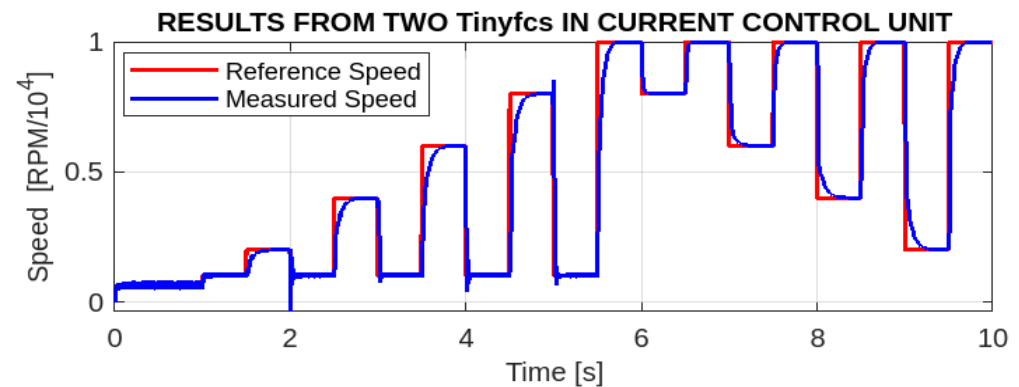


Figure 21. Results from inserting two TinyFCs in the current control unit.

Deployability remains a concern with regards to the results in Table 5. The motor control in this study is modulated at 30 KHz, thus a sampling period of 3.33×10^{-5} s. This implies that for every input signal, the control sequence should be completed within approximately 33 μ s. However, the inference time observed from deploying these models on the MCUs via the ST Edge AI Developer Cloud [46] clearly implies that the TinyFC models have a higher execution time, with the closest being the HPO model, with roughly twice the execution time of the PI-based FOC loop. From analysis, constructing the same TinyFC architecture using a different activation function like the rectified linear unit (ReLU) shortens the inference time by half since ReLU [50] uses a [0 1] activation range, rather than [−1 1] typical to Tanh activation. The same problem is observed when quantizing the model: instead of the inference time reducing, it drastically increases due to the presence of both negative and positive values in the activations. Though Tanh activation has these flaws, it is still one of the most adequate activation functions for NN-based motor control.

7.4. Contribution

The results detailed in Sections 7.1 and 7.2 underscore the significant advantages of integrating TinyFC within the control framework, particularly in the speed and current control units. The simulation outcomes, summarized in Table 4, demonstrate a marked enhancement in the performance of PI-based control when augmented with TinyFC. Notably, the TinyFC effectively eliminates the overshoot observed in the PI-based control system as illustrated in Figure 6 when applied to the speed control unit. This performance improvement highlights the transformative potential of incorporating lightweight neural networks into conventional control strategies.

Upon acceleration [51], the devised model can be fully deployed in a real-time setting, with less memory requirement as compared to the advanced techniques listed in Section 2. This can also be verified from the results in Table 5, which reports the latency of the devised and optimized NNs. From this table, on the EVBE3000D board, the devised model's inference time is ≈ 2.7 times the sampling period T_s used for PWM during the simulation experiment in Simulink. This hints that upon acceleration (sparse computing, asynchronous execution or the use of AI specific chips), the model can reach a satisfactory inference time, beneficial for EVs control.

Moreover, the observations depicted in Figure 22 demonstrate a notable reduction in the spikes that occur during the control sequence. The corrective influence of the NN effectively smooths the transitions in both the direct and quadrature currents, outperforming the performance of standalone PI controllers. These results highlight the advantages of

integrating tinyNNs into control loops, not only in terms of improved performance but also due to their computational efficiency compared to many optimal control schemes, with HPO significantly reducing the MACC number of the designed model, enhancing its practicality for real-time applications.

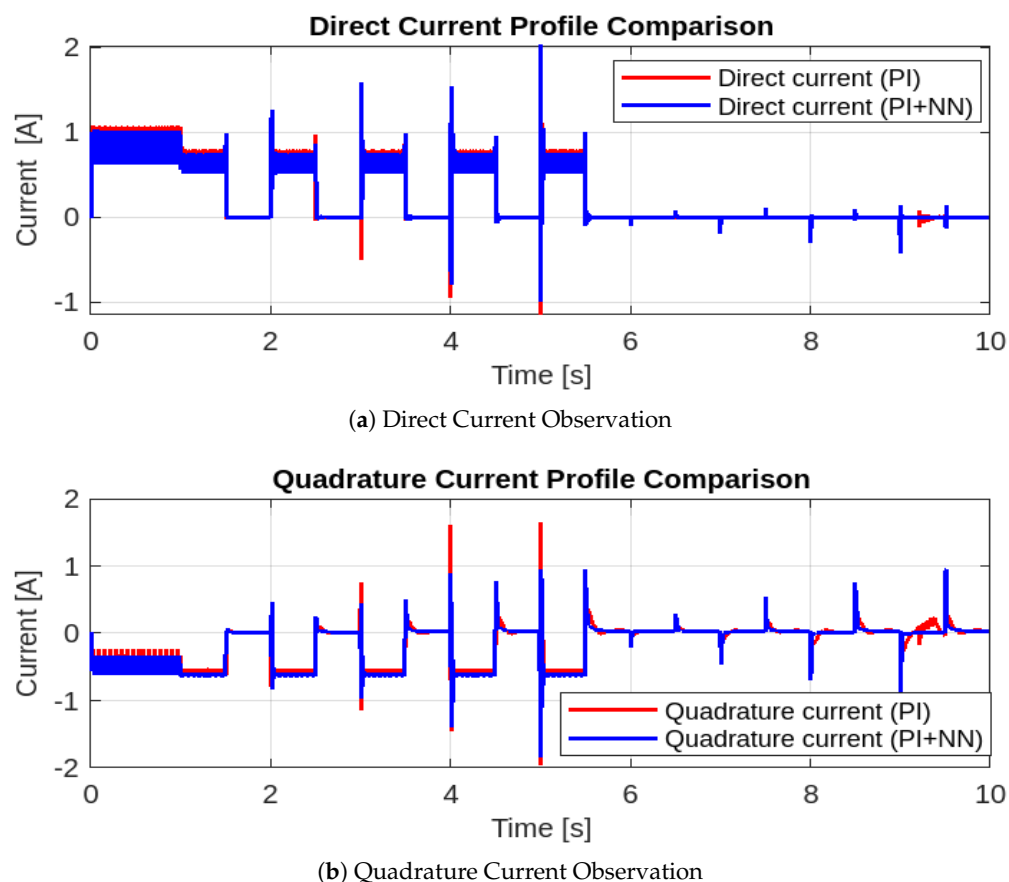


Figure 22. Current profile comparison between PI-based and NN-augmented control.

7.5. Limitations and Future Work

Despite the improvements achieved in this study, the tinyNNs used to enhance FOC still struggle with undershoots, though in very short appearances. This suggests that the trained weights and activations lack sufficient capacity to completely capture the complex dependencies between the input sequence and the output. As mentioned in Section 7.1, the MSE is used to measure the distance between the predicted value and the ground truth. Despite the MSE being relatively small, there is still a divergence between the predicted value and the ground truth value, which when averaged could hide intervals in which the model struggles to properly learn. Moreover, slight deviations in the predicted values and noise in the dataset could contribute to false predictions, which in this case is not properly addressed during training. Thus, this limitation underscores that, while TinyFC offers advantages in simplicity and deployability, it may require further refinement to meet the precision demands of high-performance motor control applications. As hinted in Section 7.3, the use of PINNs could greatly enhance the ability of the TinyFC during training. In addition to the regular learning pattern, PINNs do incorporate partial derivatives obtained from the system's equations in the loss formulation. This enhances the loss function by monitoring how well the predicted value fits in the system's physical model. The trained model is therefore guaranteed to predict values abiding to the physical equations concerned (4).

Moreover, the tinyNN model developed in this study still relies on the PI controller's output as an initial condition, limiting its potential to function as a standalone, fully

autonomous control system. As a future prospective, a standalone approach using a tinyNN to replace the three PI controllers in the FOC subsystem could be devised, thereby solely depending on the tinyNN to correct the direct and quadrature components of the voltage.

8. Conclusions

This research highlights the potential of TinyFC to enhance FOC by reducing the overshoots and tracking errors commonly associated with traditional PI-based control. By utilizing valuable sensor feedback and the PI controller's output, the proposed TinyFC model demonstrates improved speed tracking and a significant reduction in steady-state errors.

Although inference on the target MCUs still introduces some latency, NN acceleration holds the potential to ensure execution within the stringent timing constraints of the target MCUs. To facilitate the real-time inference of TinyFC within the augmented FOC framework, prioritizing a scale factor of approximately one-fifth of the sampling time is recommended for optimal results. Thus, future advancements should prioritize the development of fully hardware accelerated TinyFC models capable of replacing PI controllers entirely. This evolution would pave the way for a unified control framework that eliminates reliance on traditional control methods. Another transformative goal would involve designing a comprehensive TinyFC model capable of substituting all three PI controllers within the FOC framework: the speed control loop and both current control unit controllers. This would establish a full TinyFC-driven system, achieving streamlined, adaptive, and high-performance control without traditional dependencies.

Author Contributions: Conceptualization, M.J.M.E. and D.P.; methodology, M.J.M.E., D.P. and S.Z.; software, M.J.M.E. and S.Z.; validation, M.J.M.E., D.P. and S.Z.; formal analysis, M.J.M.E., D.P., S.Z. and T.F.; investigation, M.J.M.E. and D.P.; resources, D.P.; data curation, M.J.M.E. and D.P.; writing—original draft preparation, M.J.M.E., D.P., S.Z. and T.F.; writing—review and editing, M.J.M.E., D.P., S.Z. and T.F.; visualization, M.J.M.E. and S.Z.; supervision, D.P.; project administration, D.P.; funding acquisition, D.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: The study did not require ethical approval.

Informed Consent Statement: Not applicable.

Data Availability Statement: The dataset used to train the TinyFC alongside the control system including the reference speed signals, the FOC subsystem and the PMSM/Inverter block are presented in this paper are available in Github at the following link <https://github.com/heixiaopengyou/TINY-ML-for-FOC-of-PMSM-20092024> (accessed on 15 April 2025).

Conflicts of Interest: Authors Martin Joel Mouk Elele and Danilo Pau were employed by the company STMicroelectronics, Italy. Author Shixin Zhuang was employed by the company Mathworks, USA. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as potential conflicts of interest.

Abbreviations

AC	Alternate Current
AI	Artificial Intelligence
BO	Bayesian Optimization
CV	Computer Vision
DC	Direct current
EdgeAI	AI in Edge systems
EI	Expected Improvement

EV	Electric Vehicle
FC	Fully connected
FLOPs	32-bit floating point operations
FOC	Field Oriented Control
GRU	Gated Recurrent Unit
HP	Hyper parameters
HPO	Hyperparameter optimization
HPT	Hyperparameter tuning
LSTM	Long Short-Time Memory
LTI	Linear time-invariant
LUT	Look-Up Table
MACC	Multiply and Accumulate
MathMul	Matrix multiplications
MCU	Micro-controller unit
MIMO	Multiple-Input Multiple-Output
ML	Machine Learning
MPC	Model Predictive Control
MSE	Mean squared error
MTPA	Maximum torque per ampere
NLP	Natural Language Processing
NN	Neural Network
PC	Principal components
PCA	Principal Component Analysis
PI	Proportional Integral
PI	Proportional Integral Derivative
PMSM	Permanent Magnet Synchronous Motor
PTQ	Post Training Quantization
PWM	Pulse Width Modulation
QAT	Quantization Aware Training
SISO	Single-Input Single-Output
SPMSM	Surface PMSM
SVPWM	Space Vector Pulse Width Modulation
TinyFC	Tiny Fully Connected
TinyML	Tiny Machine Learning
TPU	Tensor Processing Unit
PINN	Physics-Informed Neural Network

References

1. Bragazzi, N.; Dai, H.; Damiani, G.; Behzadifar, M.; Martini, M.; Wu, J. How Big Data and Artificial Intelligence Can Help Better Manage the COVID-19 Pandemic. *Int. J. Environ. Res. Public Health* **2020**, *17*, 3176. [\[CrossRef\]](#) [\[PubMed\]](#)
2. Aidos, F.E.E.; Kassab, M.; Benamar, N.; Falah, B. A Comprehensive Survey on Blockchain-Based Solutions to Combat COVID-19 Pandemic. *Int. J. Comput. Digit. Syst.* **2022**, *11*, 873–892. [\[CrossRef\]](#) [\[PubMed\]](#)
3. Abadade, Y.; Temouden, A.; Bamoumen, H.; Benamar, N.; Chtouki, Y.; Hafid, A.S. A Comprehensive Survey on TinyML. *IEEE Access* **2023**, *11*, 96892–96922. [\[CrossRef\]](#)
4. Strubell, E.; Ganesh, A.; McCallum, A. Energy and Policy Considerations for Modern Deep Learning Research. *Proc. AAAI Conf. Artif. Intell.* **2020**, *34*, 13693–13696. [\[CrossRef\]](#)
5. Pau, D.; Ambrose, P. Automated Neural and On-Device Learning for Micro Controllers. In Proceedings of the 2022 Mediterranean Electrotechnical Conference (MELECON), Palermo, Italy, 14–16 June 2022; pp. 758–763. [\[CrossRef\]](#)
6. Shafique, M.; Theocharides, T.; Reddy, V.J.; Murmann, B. TinyML: Current Progress, Research Challenges, and Future Roadmap. In Proceedings of the 2021 58th ACM/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 5–9 December 2021; pp. 1303–1306. [\[CrossRef\]](#)
7. Warden, P.; Situnayake, D. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*; O'Reilly: Sebastopol, CA, USA, 2020. Available online: <https://books.google.it/books?id=sB3mxQEACAAJ> (accessed on 12 January 2025).

8. Branco, S.; Ferreira, A.G.; Cabral, J. Machine Learning in Resource-Scarce Embedded Systems, FPGAs, and End-Devices: A Survey. *Electronics* **2019**, *8*, 1289. [CrossRef]
9. Lin, J.; Chen, W.-M.; Lin, Y.; Cohn, J.; Gan, C.; Han, S. MCUNet: Tiny Deep Learning on IoT Devices. *arXiv* **2020**, arXiv:2007.10319. Available online: <https://arxiv.org/abs/2007.10319> (accessed on 12 January 2025).
10. Osekar, P.; Kalligudd, S.; Angadi, S.; Raju, A.B. Field Oriented Control of Surface-mount PMSM using Model Predictive Current Control. In Proceedings of the 2022 IEEE North Karnataka Subsection Flagship International Conference (NKCon), Vijayapura, India, 20–21 November 2022; pp. 1–5. [CrossRef]
11. Kongchoo, N.; Santiprapan, P.; Jindapetch, N. A Mathematical Model and PI Controller Design Based on Indirect Vector Control for Permanent Magnet Synchronous Motor. *ECTI Trans. Comput. Inf. Technol. (ECTI-CIT)* **2022**, *16*, 260–267. [CrossRef]
12. Cai, R.; Zheng, R.; Liu, M.; Li, M. Optimal selection of PI parameters of FOC for PMSM using structured H-infinity-synthesis. In Proceedings of the IECON 2017—43rd Annual Conference of the IEEE Industrial Electronics Society, Beijing, China, 29 October–1 November 2017; pp. 8602–8607. [CrossRef]
13. Han, J. From PID to Active Disturbance Rejection Control. *IEEE Trans. Ind. Electron.* **2009**, *56*, 900–906. [CrossRef]
14. Ekaputri, C.; Syaichu-Rohman, A. Model predictive control (MPC) design and implementation using algorithm-3 on board SPARTAN 6 FPGA SP605 evaluation kit. In Proceedings of the 2013 3rd International Conference on Instrumentation Control and Automation (ICA), Bali, Indonesia, 28–30 August 2013; pp. 115–120. [CrossRef]
15. Pingan, H.; Jagannathan, S. Adaptive critic neural network-based controller for nonlinear systems with input constraints. In Proceedings of the 42nd IEEE International Conference on Decision and Control, Maui, HI, USA, 9–12 December 2003; Volume 6, pp. 5709–5714. [CrossRef]
16. Krause, P.C.; Wasynczuk, O.; Sudhoff, S.D. Reference-Frame Theory. In *Analysis of Electric Machinery and Drive Systems*, 3rd ed.; John Wiley & Sons, Ltd.: Hoboken, NJ, USA; 2013; Chapter 3, pp. 86–120. [CrossRef]
17. Dentler, J. Real-time Model Predictive Control of Cooperative Aerial Manipulation. Ph.D. Thesis, University of Luxembourg, Luxembourg, 2018. [CrossRef]
18. Cortes, P.; Kazmierkowski, M.P.; Kennel, R.M.; Quevedo, D.E.; Rodríguez, J. Predictive Control in Power Electronics and Drives. *IEEE Trans. Ind. Electron.* **2008**, *55*, 4312–4324. [CrossRef]
19. Islam, R.; Husain, I.; Fardoun, A.; McLaughlin, K. Permanent-magnet synchronous motor magnet designs with skewing for torque ripple and cogging torque reduction. *IEEE Transac. Ind. Appl.* **2009**, *45*, 152–160. [CrossRef]
20. Grzegorz, B.; Marian, B. Developments in the Control Loops Benchmarking. In *New Approaches in Automation and Robotics*; I-Tech Education and Publishing: Vienna, Austria, 2008; ISBN 978-3-902613-26-4. [CrossRef]
21. Zhang, W.; Teng, Y.; Wei, S.; Xiong, H.; Ren, H. The robust H-infinity control of UUV with Riccati equation solution interpolation. *Ocean Eng.* **2018**, *156*, 252–262. [CrossRef]
22. Shah, B.; Bhavsar, H. Time Complexity in Deep Learning Models. *Procedia Comput. Sci.* **2022**, *215*, 202–210. [CrossRef]
23. Nustes, J.C.; Pietro Pau, D.; Gruosso, G. Field oriented control dataset of a 3-phase permanent magnet synchronous motor. *Data Brief* **2023**, *47*, 109002. [CrossRef] [PubMed]
24. Nustes, J.C.; Pietro Pau, D.; Gruosso, G. Modelling the Field Oriented Control applied to a 3-phase Permanent Magnet Synchronous Motor. *Softw. Impacts* **2023**, *15*, 100479. [CrossRef]
25. STMicroelectronics. Motor Control Nucleo Pack with NUCLEO-F302R8 and X-NUCLEO-IHM07M1. STMicroelectronics Website. Available online: <https://www.st.com/en/evaluation-tools/p-nucleo-ihm001.html> (accessed on 12 January 2025).
26. Ogata, K. *Discrete-Time Control Systems*; Prentice Hall: Sydney, Australia, 1987.
27. Akpunar, A.; Iplikci, S. Control and Parameter Estimation of PMSM by Runge-Kutta Model Based Predictive Control. In Proceedings of the 2021 12th International Symposium on Advanced Topics in Electrical Engineering (ATEE), Bucharest, Romania, 25–27 March 2021; pp. 1–6. [CrossRef]
28. Akpunar, A.; Iplikci, S. Runge-Kutta Model Predictive Speed Control for Permanent Magnet Synchronous Motors. *Energies* **2020**, *13*, 1216. [CrossRef]
29. MathWorks. fmincon (Find Minimum of Constrained Nonlinear Multivariable Function). 2024. Available online: <https://it.mathworks.com/help/optim/ug/fmincon.html> (accessed on 12 January 2025).
30. STMicroelectronics. STM32 Nucleo-64 Development Board with STM32G474RE MCU. Available online: <https://www.st.com/en/evaluation-tools/nucleo-g474re.html> (accessed on 12 January 2025).
31. David, E.; Puentes, G.; Carlos, J.; Barrios, H.; Philippe, O.; Navaux, A. Hyperparameter Optimization for Convolutional Neural Networks with Genetic Algorithms and Bayesian Optimization. In Proceedings of the 2022 IEEE Latin American Conference on Computational Intelligence (LA-CCI), Montevideo, Uruguay, 22–25 November 2022; pp. 1–5. [CrossRef]
32. Candelieri, A. A Gentle Introduction to Bayesian Optimization. In Proceedings of the 2021 Winter Simulation Conference (WSC), Phoenix, AZ, USA, 13–17 December 2021; pp. 1–16. [CrossRef]

33. Sinha, S.; Choudhary, S.; Paul, A. A Novel Two-Step Bayesian Hyperparameter Optimization Strategy for DoS Attack Detection in IoT. In Proceedings of the 2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), Bengaluru, India, 4–6 January 2024; pp. 112–119. [\[CrossRef\]](#)
34. MathWorks. Bayesian Optimization. 2024. Available online: <https://uk.mathworks.com/help/stats/bayesopt.html> (accessed on 12 January 2025).
35. Riera, M.; Arnau, J.M.; González, A. DNN pruning with principal component analysis and connection importance estimation. *J. Syst. Archit.* **2022**, *122*, 102336. [\[CrossRef\]](#)
36. Bhagavatula, R.; Savvides, M. PCA vs. automatically pruned wavelet-packet PCA for illumination tolerant face recognition. In Proceedings of the Fourth IEEE Workshop on Automatic Identification Advanced Technologies (AutoID'05), Buffalo, NY, USA, 17–18 October 2005; pp. 69–74. [\[CrossRef\]](#)
37. MathWorks. Quantization, Projection, and Pruning. 2024. Available online: <https://uk.mathworks.com/help/deeplearning/quantization.html> (accessed on 12 January 2025).
38. Woss, A. Compressing Neural Networks Using Network Projection. In *2023 Technical Articles, MathWorks*; MathWorks: Boston, MA, USA, 2023.
39. Xie, L.; Zhang, Q.-L.; Guo, M.; Wang, S.-Q. Linear pruning techniques for neural networks-based on projection latent structure. In Proceedings of the SMC'03 Conference Proceedings. 2003 IEEE International Conference on Systems, Man and Cybernetics. Conference Theme—System Security and Assurance (Cat. No.03CH37483), Washington, DC, USA, 8 October 2003; Volume 2, pp. 1304–1309. [\[CrossRef\]](#)
40. Chen, Y.; Hu, S.; Chen, D. Fast pruning strategy for neural network size optimization and its applications. *J. Chem. Ind. Eng.* **2001**, *52*, 522–526.
41. Yang, J.; Shen, X.; Xing, J.; Tian, X.; Li, H.; Deng, B.; Huang, J.; Hua, X.-S. Quantization Networks. In Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 15–20 June 2019; pp. 7300–7308. [\[CrossRef\]](#)
42. Wang, Z.; Yao, Y.; Zhou, Y. Progress of neural network quantization algorithm. In Proceedings of the 2021 International Conference on Electronic Information Engineering and Computer Science (EIECS), Changchun, China, 17–19 December 2021; pp. 411–417. [\[CrossRef\]](#)
43. Jiang, C. Efficient Quantization Techniques for Deep Neural Networks. In Proceedings of the 2021 International Conference on Signal Processing and Machine Learning (CONF-SPML), Stanford, CA, USA, 14 November 2021; pp. 271–277. [\[CrossRef\]](#)
44. Hounie, I.; Elenter, J.; Ribeiro, A. Neural Networks with Quantization Constraints. In Proceedings of the ICASSP 2023—2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Rhodes Island, Greece, 4–10 June 2023; pp. 1–5. [\[CrossRef\]](#)
45. Lee, E.; Lee, M.; Kim, S.; Lee, S.; Jang, S.-J.; Lee, S.-S. Optimization Method and Implementation of Fake Quantization from the Perspective of Hardware Performance. In Proceedings of the 2023 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC), Jeju, Republic of Korea, 25–28 June 2023; pp. 1–2. [\[CrossRef\]](#)
46. STMicroelectronics. ST Edge AI Developer Cloud. 2024. Available online: <https://stedgeai-dc.st.com/home> (accessed on 12 January 2025).
47. Bickel, P.; Doksum, K. *Mathematical Statistics: Basic Ideas and Selected Topics*; American Statistician—AMER STATIST: New York, NY, USA, 2007; Volume 56, p. 20. [\[CrossRef\]](#)
48. Alkhadhr, S.; Liu, X.; Almekkawy, M. Modeling of the Forward Wave Propagation Using Physics-Informed Neural Networks. In Proceedings of the 2021 IEEE International Ultrasonics Symposium (IUS), online, 11–16 September 2021; pp. 1–4. [\[CrossRef\]](#)
49. Trask, N.; Huang, A.; Hu, X. Enforcing exact physics in scientific machine learning: A data-driven exterior calculus on graphs. *J. Comput. Phys.* **2022**, *456*, 110969. [\[CrossRef\]](#)
50. Kaloev, M.; Krastev, G. Comparative Analysis of Activation Functions Used in the Hidden Layers of Deep Neural Networks. In Proceedings of the 2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Ankara, Turkey, 11–13 June 2021; pp. 1–5. [\[CrossRef\]](#)
51. Wu, R.; Guo, X.; Du, J.; Li, J. Accelerating Neural Network Inference on FPGA-Based Platforms—A Survey. *Electronics* **2021**, *10*, 1025. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.